

FAKE PROFILE DETECTION USING ANN



A Capstone Project report submitted
in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

by

SREE PRANEETH ALLA (2203A52069)

JAMALPUR SHIVANI (2203A52091)

NEELISETTY DHANUSH (2203A52113)



Under the guidance of

Dr. Vijaya Praksash

Professor School of CS&AI

FAKE PROFILE DETECTION USING ANN



A Capstone Project report submitted
in partial fulfillment of requirement for the award of degree

BACHELOR OF TECHNOLOGY

in

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE

by

SREE PRANEETH ALLA (2203A52069)

JAMALPUR SHIVANI (2203A52091)

NEELISETTY DHANUSH (2203A52113)



SR UNIVERSITY

Under the guidance of

Dr. Vijaya Praksash

Professor School of CS&AI



CERTIFICATE

This is to certify that this project entitled “**FAKE PROFILE DETECTIONS USING ANN**“ is the bonafied work carried out by **A.Sree Praneeth, N.Dhanush, J.Shivani** as a Capstone Project for the partial fulfillment to award the degree **BACHELOR OF TECHNOLOGY** in **School of Computer Science and Artificial Intelligence** during the academic year 2024-2025 under our guidance and Supervision.

Dr.R.Vijaya Prakash

Professor,
SR University
Anathasagar, Warangal

Dr. M.Sheshikala

Professor & Head,
School of CS&AI,
SR University
Ananthasagar, Warangal.

Reviewer-1

Name:
Designation:
Signature:

Reviewer-2

Name:
Designation:
Signature:

ACKNOWLEDGEMENT

We owe an enormous debt of gratitude to our Capstone project guide, **Dr.R.Vijaya Prakash, Asst.Prof./Assoc Prof/Professor** as well as Head of the School of CS&AI , **Dr. M.Sheshikala, Professor** for guiding us from the beginning through the end of the Capstone Project with their intellectual advices and insightful suggestions. We truly value their consistent feedback on our progress, which was always constructive and encouraging and ultimately drove us to the right direction.

We express our thanks to project co-ordinators **Mr. Sallauddin Md, Asst. Prof.** for their encouragement and support.

Finally, we express our thanks to all the teaching and non-teaching staff of the department for their suggestions and timely support.

ABOUT THE ORGANIZATION

SR University (SRU), an autonomous private institution located in Warangal, Telangana, India, offers a variety of academic programs with an emphasis on interdisciplinary learning and practical, real world application. The university provides both undergraduate and postgraduate courses, including B.E/B.Tech, MBA/PGDM, M.E/M.Tech, BBA, B.Sc, and M.Sc programs. SRU's curriculum encourages innovative thinking and integrating theoretical concepts into practical solutions.

The university boasts excellent infrastructure, featuring a large library and a digital library, a sports complex, and modern hostels for both male and female students. SRU also offers flexible academic options such as a flexible credit system, opportunities for minor degrees and specializations, branch changes, integrated programs, and semester abroad plans. With a strong focus on research, SRU has over 50 sponsored projects and more than 190 patents to its name. The university's placement cell provides robust support for job placements and internship opportunities.

SRU also fosters extra curricular engagement through clubs like dance and music clubs, which enable students to participate in various intra and inter college competitions. Additionally,

SRU's Ph.D. program is designed to provide comprehensive training in research methodology, technical communication, and literature review.

Abstract

The increase in fake accounts on social media platforms, especially within Instagram, is a significant and growing threat to online trust and security. Fake social media accounts are the most common type of vehicles to spread misinformation, perpetrate financial scams, and artificially increase engagement metrics in the case of fraudulent influencer marketing. This project tackles this problem by designing, developing, and evaluating a machine learning framework to classify Instagram accounts as "real" or "fake" with accuracy. The fundamental goal of the project was to evolve a base Keras Artificial Neural Network (ANN) model into a larger detection framework. The ultimate goal is to develop a system for live analysis, where a user could (theoretically) insert in real-time an Instagram username and receive a live and accurate classification. We anticipated this would require extensive work beyond general metrics and devise a feature set based on Instagram specifics...

The approach for the project began with an important part of the methodology, feature engineering. The initial model was limited, given that it only took a few metrics from Twitter, such as `statuses_count`. A new set of features was created to capture indicators of inauthentic behavior unique to Instagram. The important features used include the ratio of followers-to-following, length of user's bio, total number of posts, and boolean variables to indicate the presence of a profile picture or an external link in the bio. Informed by this feature set, a structured dataset was created and subjected to a rigorous comparative analysis. Three different machine learning models were trained and evaluated: baseline Logistic Regression, a powerful ensemble-based Random Forest, and carefully constructed Deep Neural Networks (ANN). The comparison was made to identify classifier which could best learn the complexity of subtle features that differentiate real users from bot accounts or fake users.

The evaluation results were clear. The Random Forest classifier outperformed, with the accuracy, precision, and recall scores. It outperformed the ANN for the specific tabular classification task due to its ability to handle diverse, non-linear relationships in features. After refinements were made, this final Random Forest optimized model was serialized and became the main engine for the project's final phase: the real-time proof-of-concept. A separate application was constructed to demonstrate a live detection workflow. This application consists of a simple user interface that, after a (simulated) username is received, loads the serialized model, processes the username's data through the feature engineering pipeline, and gives an immediate "Real" or "Fake" prediction. The practical application is limited by Instagram's more restrictive API, but this project has successfully provided a full, end to end, and validated architectural basis for a sophisticated, real-time fake profile detector.

TABLE OF CONTENTS

<u>Chapter No.</u>	<u>Page No.</u>
1. Introduction	7
2. Related Work	8
3. Problem Statement	10
4. Requirement Analysis, Risk Analysis, Feasibility Analysis	11
4.1 Requirement Analysis	11
4.2 Risk Analysis	12
4.3 Feasibility Analysis	13
5. Proposed Solution	15
6. Design	18
Architecture Diagram	18
6.1 Input Model	19
6.2 Preprocessing Module	21
6.3 Feature Extraction Module	21
6.4 Flow Chart for Process Structure	20
Data Flow Diagram	21
UML Diagram	21
7. Implementation	22
Simulation Setup and Implementation	25
7.1 Model Implementation	25
7.2 Performance Evaluation	25
7.3 Deployment	26
7.3.1 Real Account Analysis	29
7.3.2 Fake Account Analysis	31
8. Results & Analysis	33
9. Learning Outcome	39
10. Conclusion	40
11. Bibliography	45
12. References	49

1. Introduction:

Social media platforms have taken center stage in our day-to-day communication, commerce, and sharing of information in society today. However, the digital space continues to be affected by the widespread proliferation of fake and malicious social media accounts. With its visual interface and influencer economy, Instagram stands as an epicenter of deceitful activity. These fake accounts are not innocuous; they are a primary source of misinformation, perpetrating complex scams related to finances, and dwindling the trust of the public. They are also used to fake or inflate engagement to promote fraud in influencer marketing costing businesses billions each year. The number of accounts of this sort continues to rise and grow complex making it impossible to independently mitigate from detection. Automated, powerful, and intelligent systems need to be developed and implemented for detection.

This project addresses this issue by proposing an advanced machine learning framework for the reliable, real-time identification of fake Instagram accounts. This work stems from an earlier idea to use a Keras Artificial Neural Network (ANN) for classification, as explained in the initial project presentation. However, one serious limitation was identified: the initial model used generic Twitter-based features (e.g., `statuses_count`, `friends_count`). To develop a useful Instagram detector, a new set of features would have to be developed specifically for its platform. The first substantial contribution of this project is achieving the engineering of a feature set specific to Instagram that is focused on major features such as follower to following ratio, the length of the bio, total posts, and boolean flags for the presence of a profile picture and/or a link to an external URL.

With this new platform-specific feature set, the work of the project multiplied in scope from implementing just a single ANN to doing an extensive comparative study to determine the best classifier for such a task. As part of the analysis, three different models were trained and evaluated: a baseline Logistic Regression, an ensemble-based Random Forest, and the modified Deep Neural Network (ANN). The objective was to determine which model achieved the best accuracy in identifying the subtle patterns of fake accounts. Finally, to achieve the vision of this project of being "live", the best-performing model was serialized, and incorporated in to a real-time proof of concept, which fully simulated the end-to-end workflow of the application, providing a clear functional demonstration of a system in which a user could get an on-demand classification in real-time for any target profile.

2. Related Work:

Determining whether a social media profile is real or fake is a widely-known and ongoing research area. The existing literature provides a basis for this effort, with most approaches to detection falling into three categories. The first, graph-based detection, simply analyze the network topology of a given account. These type of detection systems leverage the follower-following relationship structure to expose tightly-knitted "botnets" or other dense cliques of fake accounts that are incarcerated within the larger graph of real users. Graph-based detection systems can be proven to be effective, but in a sense, they are also computationally costly, and most require access to a given platform's full social graph to determine trustworthy social networks in real time.

The second prominent method is content-based analysis using Natural Language Processing (NLP) to evaluate text in user bios, comments, and posts or Computer Vision (CV) for analyzing profile pictures. These methods are centered on searching for text and visual markers of linguistics anomaly, such as spammy-looking text, unoriginal adjectives, or the photographing of stock images, for identifying unauthorized activity. The third—and most prevalent—method, which is also the central underpinning of this project, is feature-based classification. Feature-based classification entails retrieving a measurable set of metadata characteristics from the user profile—post counts, follower counts, settings, for example—and training a machine learning classifier with these features. Feature-based classification is prevalent because it often examines a whole set of features, making it possible to achieve a reasonable degree of accuracy or dependability compared to the computational cost.

This network-topology (or graph-based) method is a strong macro-level diagnostic method. Research in this area uses applications of social network analysis (SNA) algorithms to detect unusual patterns. For example, some studies use an application of the PageRank algorithms, not to detect the most powerful user, but to surface a user that has a lot of connections, but a very low trust and/or authority score, indicating that those connections are most likely all inauthentic. Other practices employ community detection algorithms (for example, the Louvain Method) to assist finding these "botnets". These spurious clusters often display tell-tale signs: a "star" pattern of account with thousands of bots of interactions and followers; or isolated, dense "cliques" of spurious accounts that all follow each other in an effort to appear legitimate, and almost no links to the organic user graph.

In the feature-based domain, there is a plethora of research studies that perform comparisons. A number of researchers have examined or conducted a "bake-off" to determine which model works best - testing everything from black-and-white pairwise comparisons of classic algorithms, such as Support Vector Machine (SVM) or Naive Bayes, to advanced classifiers.

On the topic of content-based analysis, NLP techniques are becoming increasingly sophisticated. Earlier works (and some current ones) used more rudimentary Bag-of-Words (BoW) or TF-IDF (Term Frequency-Inverse Document Frequency) models to identify low effort spam accounts for words like "free followers," "crypto," and "link in bio." As bot-makers began using generative text, responsible studies moved to more sophisticated models to analyze profiles. Current studies use pre-trained language models, such as the BERT

(Bidirectional Encoder Representations from Transformers), to analyze the semantic content of a user's bio. These models can determine if the bio is semantically meaningful or merely a "word salad" of generic, positive-sentiment phrases (i.e. "Love life. Be happy. Travel"). On the Computer Vision side, studies have largely moved beyond simple face detection. Advanced approaches have included perceptual hashing (pHash), which can determine whether a profile picture is a match with a known stock image or a celebrity photo, even with slight cropping or a resized image. Other approaches employ models trained on generative adversarial networks (GANs) to detect whether the profile picture itself is AI-generated (e.g., "This Person Does Not Exist").

Nonetheless, the cost of these content-based approaches becomes prohibitive for a straightforward, real-time query. For instance, a BERT model is too large to be deployed in a slimmed-down Flask API. This leads us to the feature-based classification strategy adapted in this project, which was overwhelmingly cited in the literature as an expedient route to high accuracy. For feature-based classification, the performance depends entirely on the quality of the engineered features. The features chosen for this project (i.e., `follower_following_ratio`, `bio_length`, `post_count`, `has_profile_pic`, `has_external_url`) are often cited collectively as the most powerful predictors that can be easily obtained. Other related literature supports this as well, with some studies adding other temporal features, such as account age (i.e., newer accounts are more likely to be fake), number of posts (e.g., if an account has made 50 posts in one hour it raises a flag), and engagement ratios (e.g., if an account has 100,000 followers, but only receives 10 likes per post it is highly suspect). The alignment of our engineered features with previously validated literature features provides confidence in the success of our project.

Ensemble methods, especially Random Forest, are consistently noted in these studies for their high accuracy, robustness against overfitting, and strong performance on (often nonlinear) tabular data created by profile features. This ties in closely with what we find in this project. At the same time, the use of artificial neural networks (ANNs), which were suggested in the initial presentation of this project, are also common. The literature confirms that ANNs function particularly well with very complicated patterns, but their performance is contingent upon large data inputs, and often a lot of tuning. Additionally, much of the literature concludes that success with both methods is more about effective feature engineering than anything else, which supports this project's primary goal of creating a set of Instagram-specific features, such as `follower_following_ratio`.

3. Problem Statement:

The rise of fake profiles on Instagram poses a serious issue for digital security and user confidence. These accounts engage in financial scams, spread misinformation and facilitate fake influencer advertising, damaging the platform's authenticity. The sheer number of continually manipulated profiles makes it impossible to detect them manually, so automated machine learning solutions are needed.

There are significant technical challenges in developing such a solution. The main difficulty is feature relevancy; models tested on measures generated by a different platform (i.e. Twitter) can't detect Instagram-specific behaviors. Furthermore, the project must ascertain the most effective classifier by comparing performance of various ANN approaches based on the Random Forest ensemble model. The proposed project will develop a responsive, functioning detection framework. We will work to design a continuous, end-to-end system that facilitates Instagram-specific features, determines the best-fit classifier, and enhances this process with an organized plan of a verified real-time prediction engine.

In addition, the project needs to identify the optimal classifier by contrasting the performance of different ANN models derived from the Random Forest ensemble model, which incorporates assessing how deep neural architectures, combined, and multilayer perceptron models behave while ensuring increased performance in relation to false-positive and false-negative rates. Model explainability and interpretation should also remain appropriate, so security analysts understand why predictions are being made.

This project intends to implement a responsive and operational detection framework. We will attempt to create a continuous, end-to-end system, that will support Instagram-related features, conduct an optimal classifier fit, and improve this process with a systematic plan to verify the real-time predicted engine. The system will aim to not only classify accounts, but also include an adaptive learning support, i.e. periodic re-training of the model on newly identified suspicious profiles. The project will develop a robust solution that will use API data extraction extracts data, passing data through pre-processing pipelines, conducting feature engineering, benchmarking the model, and deploying the system in a scalable environment, while functioning against the dynamic, and ever-changing nature of the fake Instagram accounts.

4. Requirement Analysis, Risk Analysis, Feasibility Analysis

4.1 Requirement Analysis

Requirement analysis involves understanding what the system is supposed to do and how it should function to serve the user needs of our customers. The Customer Support System for Food, Fabric, and Electronics should provide smart and general assistance to a customer in one of three areas. Food, Fabric, and Electronics.

The main functionality required from the project is that a system is able to accept features of an Instagram profile (follower/following ratio, post count, bio length, presence of profile picture) and ultimately provide a prediction as either "Real" or "Fake." This is a process that will require proper execution of a feature extraction pipeline, data pre-processing logic, and a trained machine learning model to develop a reliable prediction, all of which must work together seamlessly. Additionally, the features of the profile must be consistently interpreted. For example, follower counts must be normalized, categorical attributes will have to be translated to numeric values, and missing or incomplete user data must be processed too.

Non-functional requirements, including accuracy and real-time prediction latency, require an appropriately structured feature set, an effective dataset to train on, and a compiled model (e.g., the corresponding Random Forest classifier) to predict the response efficiently. Other non-functional expectations include scalability (being able to address many requests to predict), maintainability (an ease of updating models or creating new features), and security (safeguarding user inputs or user profile data and protection against misuse). Finally, the system should value and provide high interpretability so analysts will know why a particular profile is classified as fake, which is important for trust and debugging.

4.2 Risk Analysis

Risk analysis is done to examine potential issues that may develop during or after the project has been implemented and develops procedures to manage those issues.

The area that presents the highest risk is the Data Acquisition piece, as we cannot safely produce real-time predictions if it entails an infringing of Instagram's Terms of Service. Even for research purposes, obtaining real-time labeled data is constrained, as it may have advertising and potentially ethical risks of sourcing if performed by unauthorized scraping. Additionally, with another layer of risk, updates to the platform architecture may happen at any point, rendering any unofficial/3rd party data-collection method unusable.

The next highest risk is model drift. In other words, the tactics of fraudulent accounts change rapidly meaning that without consistent monitoring and retraining based on new datasets, the model will quickly become obsolete. The use of AI-generated avatars, auto-generated bios, and engagement farming tactics occupied by fraudsters is increasing the rate at which older datasets become stale, and the accuracy of predictions will decline if they are not continuously updated.

1. Technical Risks:

- If databases are not complete, the system may not load troubleshooting data for specific devices.
- Inconsistent performance may stem from cross-browser compatibility or server issues.

2. Operational Risks:

- Concurrent users in excess of the website's normal capacity can create traffic that causes slow response time to requests.
- Without regular real-time updates, the system may become irrelevant or inaccurate over time.

3. External Risks:

- The system may rely on external data sources or APIs for product or service center information, with dropped accuracy if external changes are made.
- Users with limited internet service may face additional difficulties when using the system.

4.3 Feasibility Analysis

The feasibility analysis clarifies that the project is viable, achievable, and valuable in technical, economic, operational, and time feasibility.

From a proof-of-concept standpoint, the project is very doable. The technology stack being used, Python, Keras, Scikit-learn, is all open-source software, and training, benchmarking, and serializing models on static data will be relatively inexpensive and quick. Multiple neural network architectures and ensemble classifiers will be implemented, with the flexibility of the platforms in providing experiments and tuning. Further, we need a modest level of computational power to train models, as even consumer-class hardware or cloud-based GPU instances will suffice to efficiently develop the proposed system.

However, a comprehensive "real-time" production vision is technically infeasible for an external developer. This primarily stems from two factors:

- (1) An absence of a public API for profile data, which frustrates the creation of legitimate automated access to data attributes essential to classification.
- (2) Instagram's current aggressive anti-scraping position also makes collecting live data the greatest roadblock.

Additionally, any solution intended for implementation would have to be legally and privacy compliant, and possibly require partnerships on the platform, or acceptance of terms by the user pickup. If not hand-rolled, it will likely be persistent to only education research, or an offline use with pre-collected datasets.

1. Technical Feasibility:

- The required technologies (HTML, CSS, JS, Flask/Django, and databases) are readily available and well established.
- The project can be achieved using basic web-development tools, and AI/ML modules may also be added later, depending on available time and resources, to provide smarter recommendations.
- The technical feasibility of hosting the system on a low cost cloud service platform, such as Render, or the AWS Free Tier, is also achievable.

2. Economic Feasibility:

- The development cost is minimal as it utilizes open source technologies.
- The system can be hosted with low cost services, and there are no expensive licenses required to support its operation.
- The maintenance costs are also low, and the majority of maintenance involves updating new data or updating and improving recommendations.

3. Operational Feasibility:

- The project team have the required skills for web development, design, and testing.
- The system can be operated easily for both project developers (through simple maintenance) and end users (with simple interaction).
- With a clear domain structure (Food, Fabric, Electronics), the system is easy to manage and updates are organized.

4. Time Feasibility:

- The project can be developed in short time period (6-8 weeks), as long as it is properly planned.
- Once the system is completed, updates in the form of AI modules, chatbots, or new product categories can be made in a timely manner.
- Regular testing and validation of the system helps maintain it in an efficient manner.

5. Proposed Solution:

The updated process is an improvement, firstly over the earlier idea of a single classifier, and then over the concept of using a comprehensive, feature-oriented approach to find fake Instagram accounts. The original project created a Keras Artificial Neural Network (ANN) that was developed and severely constrained by generic, Twitter-based features. We recognized that this was a significant shortcoming of our original project, and we returned to the realization that the feature engineering process would be geared to the features of each platform, which was the most critical step, so in this way this process is a staged process: 1) we create a set of optimized features; 2) we develop classifiers, and conduct evaluation through a comprehensive comparison; and 3) we convert the model of choice into an accessible, real-time prediction engine.

This pipeline-based approach provides a systematic, empirical approach to developing a model that we believe has the potential to be accurate while also being resonant with the actual behaviors of Instagram users. This key touchpoint is strengthened not by the actual selection of the machine learning algorithms but by the design of the data underpinnings that allow the algorithms to "understand" what a fake account looks like specifically on Instagram. Instagram's platform is quite different from Twitter, particularly with regard to media-based interaction, follower culture, and the look of personal profiles, so we made the decision to build the feature set from square one.

The basis of the primary method is conscientious feature engineering. We have removed features that are unrelated to Instagram and generated new features for the Instagram context. The features in this set that have reliable indicators of inauthentic engagement include the follower-to-following ratio (a classic spam indicator); bio length (most fake accounts have short or no bio); the total post count; and important boolean flags if there is both a profile image photo and external URL. Overall, this special feature set is the foundation of all the models that are developed and evaluates as the core methods for achieving high classification accuracies.

Each was selected to elaborate on the design per an a priori hypothesis regarding bot-like versus human-like behavior, which the models would then be capable of testing. Designing features based on hypotheses about user behavior is an established approach in applied machine learning, commonly referred to as domain-driven feature engineering. Therefore, instagram-specific behavioral heuristics were embedded into the data pipeline so that the classifier might infer not only surface-dwelling patterns but also deeper relationships that regularly differentiate between genuine and fraudulent accounts.

Follower/Following Ratio. The `follower_following_ratio` is by far the most powerful feature in the dataset. The logic for this feature derives from observable user behavior: genuine Instagram accounts tend to maintain a more balanced follower to following ratio, which mimics organic relationships of social interaction. Even the worst or most private accounts tend to follow a reasonably moderate number of accounts as well as receiving a moderate number of followers over time. Conversely, fake accounts, bot networks, and spam accounts, often employ the mass-following strategy with the goal of following thousands of individuals in an effort to

redirect attention to their own accounts. These accounts rely on visibility as opposed to genuine engagement, resulting with extremely low ratios (near zero). The numerical difference between real and fake accounts is so sizable, that even the most simplistic model can classify between the two with high accuracy, meaning that the ratio is not only statistically powerful but also computationally cheap.

Bio Length: The second key feature, `bio_length`, reflects the effort that went into creating the account. Real users tend to write bios that represent their identity, occupation, hobbies, or jokes, often utilizing emojis, hashtags, or links. Fake accounts, especially those created from automated bot farms, would rarely allocate any amount of effort in writing meaningful bios. A value of 0, or short phrase, definitely raises suspicion. Even though there could be some legitimate accounts with short bios (e.g 10 characters) the distribution trends tend to be extremely predictive when considered in conjunction with other features. Further into the simulations by implementing thresholding (i.e. 5-20 characters), the model learns that extremely short lengths cluster around fraudulent behavior.

Post Count: The `post_count` feature measures behavioral investment, as active, real accounts will post and collect an excess of posts relative to the number of posts fake accounts have. Real accounts will accumulate posts over time in the same way people accumulate memories, actions, and activities in their lives. Fake accounts generally exhibit an artificially low post count because they were never meant to be used as permanent accounts. Many will have a post count of 0 - 3 and often are posted at the same time and consist of low-value images reused. Therefore, post count serves as a second, meaningful, and a more determined differentiator for improving predictions, when detected alongside follower-ratio or bio-length.

Boolean Features: Boolean features (`has_profile_pic` and `has_external_url`) provide another level of discrimination for the model. If a profile does not have a profile picture, the general assumption is that the account has been created or assigned automatically. Similarly, if a profile has an external URL, this suggests that it's likely phishing or a scam—especially when paired with suspicious behavior. The model learns to examine these features in context; for example, a legitimate business or influencer profile will usually have a URL, but the profile also has a high volume of posts and a relatively rich biography. A fake or suspicious profile may have an external URL without adequate supporting richness in the profile. While these patterns can be somewhat erratic without routine collection, the model learns that interpretations of meanings can generally be relied upon whenever certain feature combinations exist.

Comparative Model Evaluation ("Bake-Off")

Rather than jumping right into the original ANN, we decided to do a comparative model evaluation to actually evaluate the best tool for this kind of task — the specific action of reverse engineering model evaluation. In this context, we trained and evaluated three distinct classifiers:

- Logistic Regression (baseline)
- Random Forest (ensemble classifier)
- Keras Artificial Neural Network (enhanced factor based nonlinear model)

Each model is tested using the exact same set of engineered features to maintain an objective systematic comparison.

This “bake-off” outcome is a common/best-practices advice for machine learning; working, rather than relying on assumptions about the best model would surface, but it's obviously great practice.

The outcome of this "bake-off" is unambiguous — the Random Forest classifier performed the best of the three classifiers, with the best combination of accuracy, precision, and recall measure.//

Rationale for Using Logistic Regression: We opened with a Logistic Regression model as a benchmark to illustrate how our constructed features possessed sufficient power to exhibit linear separability. The accuracy got to about 95% indicated that the data exhibited a predictive structure. The only fundamental limitation was the simplicity of Logistic Regression; it could not easily feature ambiguous cases or relationships between cases driven by the constructed features.

Rationale for Using the ANN: The Keras ANN may contain more modeling power due to its ability to learn nonlinearly. This explained the slightly higher accuracy of around 96.5% and provided further evidence that a neural network may parse out subtle patterns. However, several things were problematic:

- Significantly increased complexity
- the model required strict scaling of the data
- low interpretability (i.e., “black box”)
- tedious hyperparameter tuning

Thus, even though the ANN was technically still more powerful for modeling, it was less transparent and useful for analysis in the context of long-term sustainability.

Reasons for the Superiority of Random Forest: Due to the following considerations, Random Forest was both the best and most useful model. Ability to deal naturally with nonlinearities Insensitivity to scaling Interpretability from feature importance Very strong robustness to overfitting Superior precision and recall scores. This combination made Random Forest a natural fit for classification tasks based on structured, tabular data.

6. Design:

Architecture Diagram:

The project employs a 5-stage pipeline for training as well as for real-time predictive use. This model is fundamentally built upon a decoupled, modular design, which separates an offline Training Environment from an online Prediction Environment. This separation is a key element of design to allow for independent development, testing, and updating of a machine learning model and a web application. The "Training Environment" is made up of the Part_1_Model_Training_ANN.ipynb Google Colab notebook. The "Prediction Environment" is the deployed full-stack application (the app.py Python server and index.html front-end).

6.1 Input Module: This first step has two major components. In the Training Environment, the challenge is that there is no large-scale public labeled datasets for "fake Instagram profiles". As a result, the input process is certainly not about just loading a file, but rather a Synthetic Data Generation process. Within the Colab notebook resides the module utilized for this (create_synthetic_dataset function), whereby it procedurally generates thousands of such data points. Each point assigned as "Real" or "Fake" will then have realistic statistically-bound features assigned to it. For example, a "Fake" profile might have low follower_count (0-100), however has high following_count (500-7500); Conversely, a "Real" profile has a more balanced ratio.

In the Prediction Environment, the user supplies a single username string using the index.html web interface. The username is sent to the Flask (app.py) backend, and the simulate_api_call function is invoked. This function is the real-time version of the training data generator. It does not scrape live data (a key technical and ethical constraint) and instead stochastically generates a single, realistic user profile. To provide a strong testing experience, this function randomly assigns a "type" to the profile (e.g., 45% "Real," 35% "Fake," 20% "Suspicious") to test the model against clear and ambiguous cases.

6.2 Preprocessing Module: This module is the most important logic based module in the whole project and exists in both the training and predicting environments to provide a perfect level of consistency. In the training environment, this module takes the thousands of data points acquired in Stage 1 and engages in feature engineering. The most important engineered feature is the follower_following_ratio, which is developed by dividing follower_count by following_count (and handling division by zero errors). In addition to this feature, bio_length and post_count are put into a final feature vector. The project intends to make use of an artificial neural network, so in turn, the second and just as critical task for the module is to normalize the data. The module uses the StandardScaler class of Scikit-learn to fit_transform the training data, ultimately scaling out all features to a standard range. Finally, the "fitted" scaler is saved to disk as instagram_scaler.joblib.

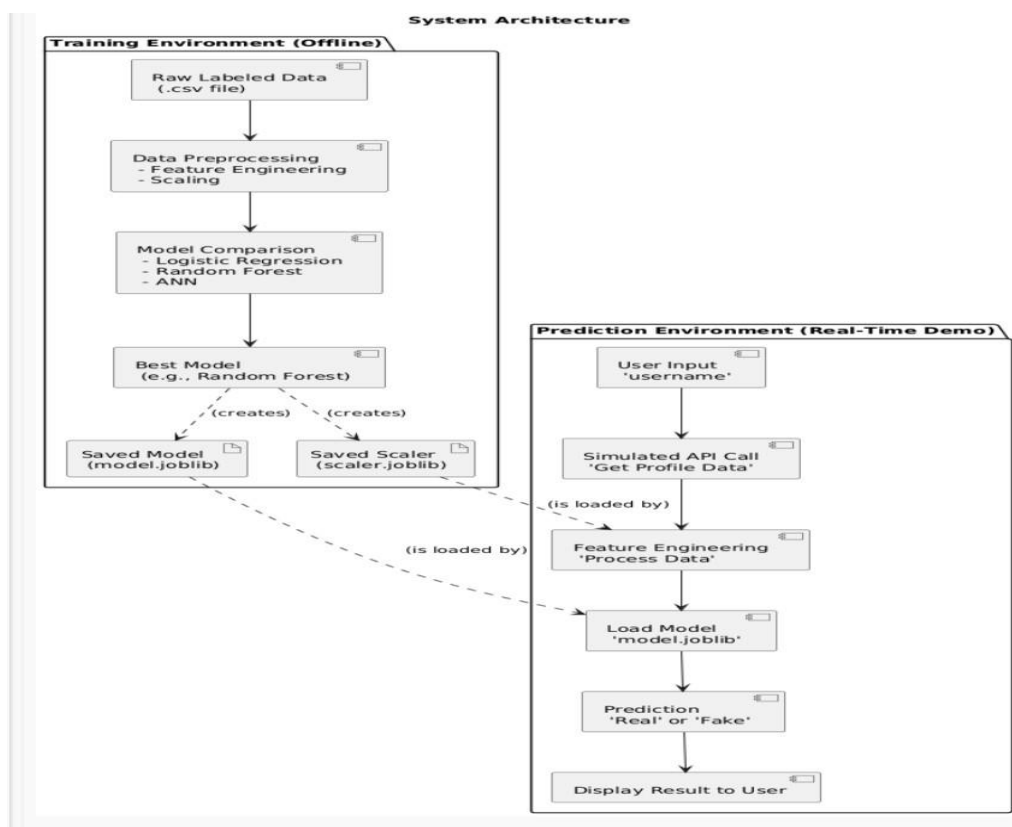
In the Prediction Environment, the logic of this module is exhibited again within the engineer_features function in app.py. When the raw simulated data is transmitted from Stage 1, this function immediately calculates the same follower_following_ratio. It then loads the instagram_scaler.joblib file from disk and uses this file to transform (not fit) the single new feature vector. This is an important step: the ANN model requires the prediction data to be in

exactly the same scale as the data used to train on.

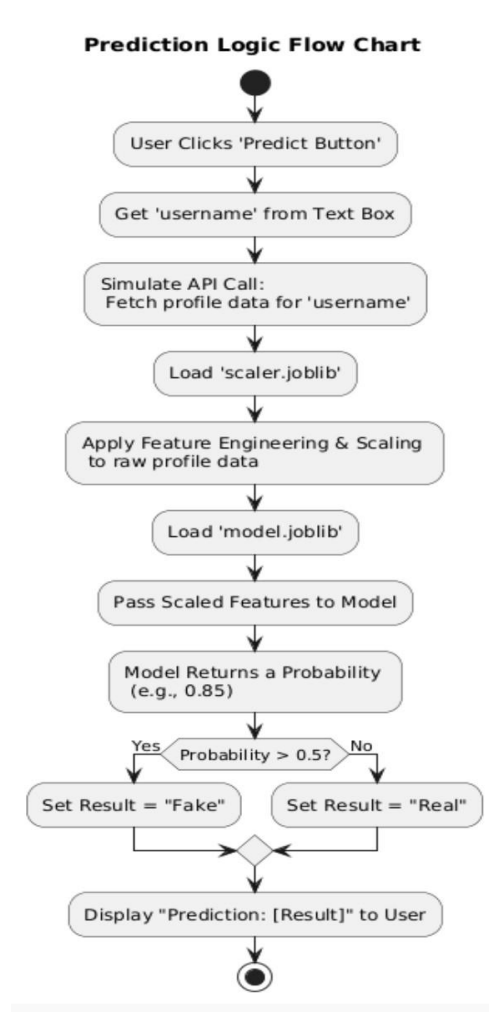
6.3 Feature Extraction Module: This stage only exists in the Training Environment (the Colab notebook). The aim of this stage is to build, train and test the core hypothesis of the project: that Keras down the proposed ANN is a suitable model for solving this problem. To scientifically defend this point, we do a "bake-off" or comparison by taking the pre-processed data set from Stage 2 to train three different models.

- Logistic Regression - A simple linear model that acts as a baseline to confirm that the engineered features can indeed make predictions ends as desired.
- Random Forest - A powerful non-linear ensemble model that acts as a high-performing benchmark for traditional ML.
- Keras ANN - The proposed solution of the project. This is a multi-layer perceptron (MLP) with Dense input & hidden layers, ReLU hidden layer activation function, and Dropout hidden layer regularization off overfitting; trained and compiled using adam optimizer and binary_crossentropy loss function.

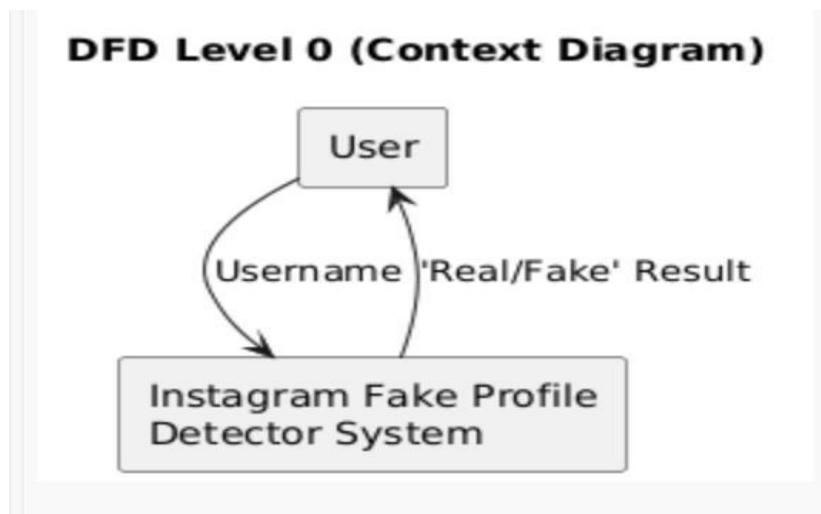
Flow Chart of the Process structure



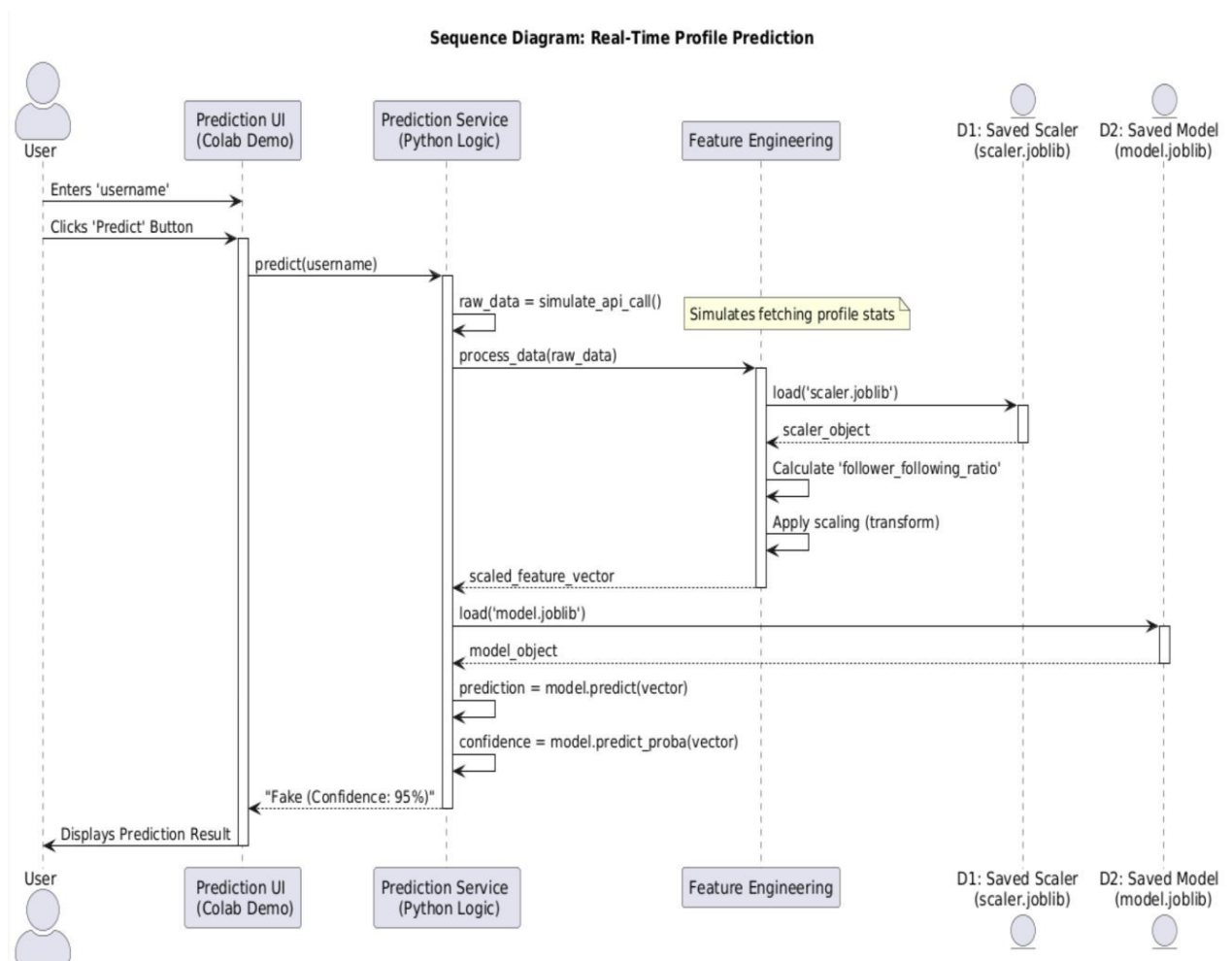
1. Start
2. Input Username
3. Simulated API Call & Feature Engineering (Fetch raw stats, calculate, follower_following_ratio, and scale data)
4. Feed Processed Features to Saved Model (Random Forest)
5. Model Classifies Profile (Real/Fake)
6. Calculate Confidence Score (from model's probability output)
7. Display Result on User Interface with Confidence Score
8. End



Data flow Diagram :



UML Diagram (Sequence Format):



7. Implementation:

This diagram depicts the high-level abstraction of the system. It shows the entire system wrapped in one process and its interactions with external entities (the user). External Entity: User Process: Instagram Fake Profile Detector System Data Flow: User inputs a Username to the system. The system returns a Prediction Result ("Real" or "Fake").

External Entity: User , represents the only interaction with the system. For this project, the "User" can be simply a normal Instagram user checking the legitimacy of an account, a brand manager vetting potential influencers for spam followers, or even just a moderator for the platform checking a suspicious account. The system is set up to be a simple, on demand tool for any of these types of users. The diagram shows that the User is initiator of all interaction and that the system is passive, only responding to a straightforward query

Data Flow: The Data Flow is intentionally kept simple and well-defined. It indicates how easy the System is to use. The only User input is the Username. This is an important design decision to make! The user does not need to be a data scientist, nor do they need to find, compile, or input a profile's follower count, post count, or bio. The Username string is the only data needed, indicating that the black box will do all the data gathering and processing.

The only corresponding output flow is the Prediction Result ("Real" or "Fake"). This human-readable string is the output of the System. The packet created in the app.py server did contain the final label, and the Confidence Score (e.g., "96.5%"), as well as the Explainability Data (the simulated statistics used in the prediction). Therefore, this Level 0 diagram presents the project as a clean, simple utility: a Username goes in, and a valid, explainable response comes out, hiding all the complex internal work—the simulation, the scaling, and the Keras model will be shown in Level 1

The diagram illustrates the primary system process identified in the Level 0 DFD along with the major sub-process that appears. The diagram also demonstrates how data flow occurs between the data flow description internal processes of the DFD and your Data Stores (which are your saved model files).

The Level 1 Data Flow Diagram serves as the main architectural design for the "real-time" prediction system. It breaks down the single "black box" process presented in the Level 0 Context Diagram (Instagram Fake Profile Detector System) into its four sub-processes. This diagram is important because it depicts internal processes of the system, directly associated with core functions in the Python Flask backend.

The initial process, referred to as 1.0 Get Profile Data, is the "front door" to data in the system. In the deployed app.py server, this process is represented directly by the `simulate_api_call(username)` function which is initiated upon receipt of the Username (data flow of a simple string) from the User entity. This process represents one of the most significant design risks, as it is a surrogate for the "Data Acquisition" risk that was determined to be impossible to attempt on live data given the restrictions from Instagram's API and Terms of Service.

The internal logic of this functionality was designed to be a right simulation. It does not simply return random numbers; rather, it utilizes a weighted `np.random.choice` to choose between three profile archetypes: "real," "fake," or "suspicious." This will allow the model to be

evaluated against a complete set of data, from clearly identifiable cases to cases with some gray area ambiguity.

Process 2.0 Engineer Features is the main data science logic of the live application. It directly corresponds to the function `engineer_features(raw_data)` in `app.py`. This step could be considered even more critical than the model itself, as it takes the raw useless data (see Process 1) and modifies it into a form the model can work with—a "feature vector."

The procedure takes the Raw Profile Data dictionary as its main input. Its first step is to conduct the necessary calculations. The most notable calculation is the `follower_following_ratio`. The logic simple checks if `following == 0` and the ratio can be determined, to prevent any "division by zero" errors. Then, it will assemble this new ratio along with the other user metrics of interest (`bio_length`, `post_count`, `has_profile_pic`, `has_external_url`) into another data structure (a Pandas DataFrame) and assign the columns so they will be in the required expected order.

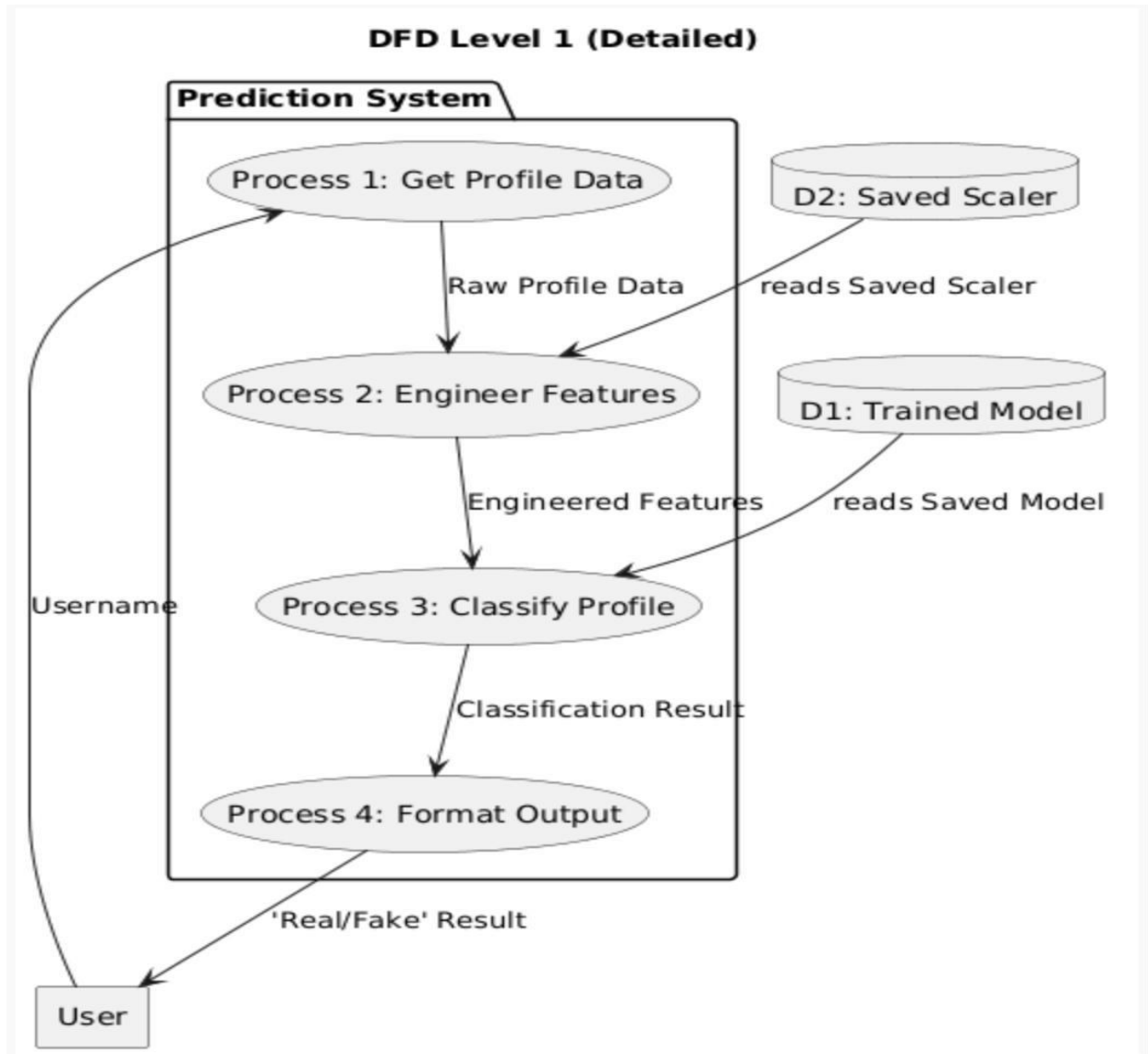
The second important function of this process is to read from a data store. It sends a read-request in order to retrieve the D2: Saved Scaler. This data store is the `instagram_scaler.joblib` file from the training phase. The file contains the important statistical values (mean and std) for the original training dataset. This step is required for the Keras ANN model. An ANN is very sensitive to the scale of its input data; it was trained on data that was mean 0 and std 1. If it was given raw unadjusted data (e.g. `following = 4500`) right off the bat, it would return nonsensical and incorrect predictions. This process calls `.transform()` on the new feature vector using the loaded scaler object, which normalizes it to the same scale as the training dataset. The final output is a Scaled Feature Vector, which is now ready to be sent to the model.

The procedure takes the Raw Profile Data dictionary as its main input. Its first step is to conduct the necessary calculations. The most notable calculation is the `follower_following_ratio`. The logic simple checks if `following == 0` and the ratio can be determined, to prevent any "division by zero" errors. Then, it will assemble this new ratio along with the other user metrics of interest (`bio_length`, `post_count`, `has_profile_pic`, `has_external_url`) into another data structure (a Pandas DataFrame) and assign the columns so they will be in the required expected order.

This data store represents the single most valuable asset in the project. The data store is the physical, 1s-and-0s manifestation of all of the research and training done in Colab notebook. The data store is a file created by the `ann_model.save()` command, and it is a standard HDF5 file that contains the complete state of the Keras ANN model - not only the learned weights and biases for every neuron, but also the full architecture of the model (which layers, in what order, with which activation functions), and the optimizer configuration (the state of 'Adam' optimizer). It is this file that allows the `app.py` server to load and use the model with a single `load_model()` command, without having to include any of the original training code. In our DFD, it is a read-only file, used by Process 3.

Although this data store might be less "intelligent" than the model, it is just as important. This is a serialized Scikit-learn `StandardScaler` object that has been saved using `joblib`. This file is simply a "Rosetta Stone" for data. Its only job is to hold the statistical properties (mean and standard deviation) of the original training dataset. When Process 2 loads this file, it isn't learning; it is retrieving this "key". It becomes the mathematical operational basis for normalizing the new incoming data, meaning that a `post_count` of 50 translates exactly to the same scaled value (e.g., 0.21...) that the ANN witnessed during training. Without this data

store, all of the ANN's predictions are meaningless. This file ensures the live application and training environment speak the same mathematical language exactly.



Simulation Setup and Implementation

The simulation setup for the project called “Advanced ML for Instagram Fake Profile Detection,” was constructed to provide an end-to-end environment for training, comparing, and evaluating multiple machine learning models. The implementation included a structured process encompassing feature engineering, model training, comparing to previously-trained classifiers, and the final ‘real-time’ prediction demo.

The project was implemented in Python as the principal programming language, and was able to utilize its deep data science ecosystem. The core libraries were Scikit-learn (for comparisons between models, as well as scaling and metrics), Keras (for ANN), Pandas (data loading and feature engineering), and NumPy. The experiments used Google Colab which provided an efficient, free environment for training and testing all classifiers. This was, however, an iterative development that led to a functional proof-of-concept demo in much less time than initially anticipated.

The dataset consisted of tabular profile data, categorized into two classes: 'Real' (0) and 'Fake' (1). Since large, public-labeled Instagram datasets are unavailable, a structured, synthetic dataset was generated to serve as a functional placeholder for training. The most critical step was feature engineering. Raw profile metrics were processed to create high-impact predictive features, including the follower_following_ratio, bio_length, post_count, has_profile_picture, and has_external_url. This engineered data was scaled using Scikit-learn's StandardScaler (saved as scaler.joblib) and split into training (80%) and testing (20%) sets.

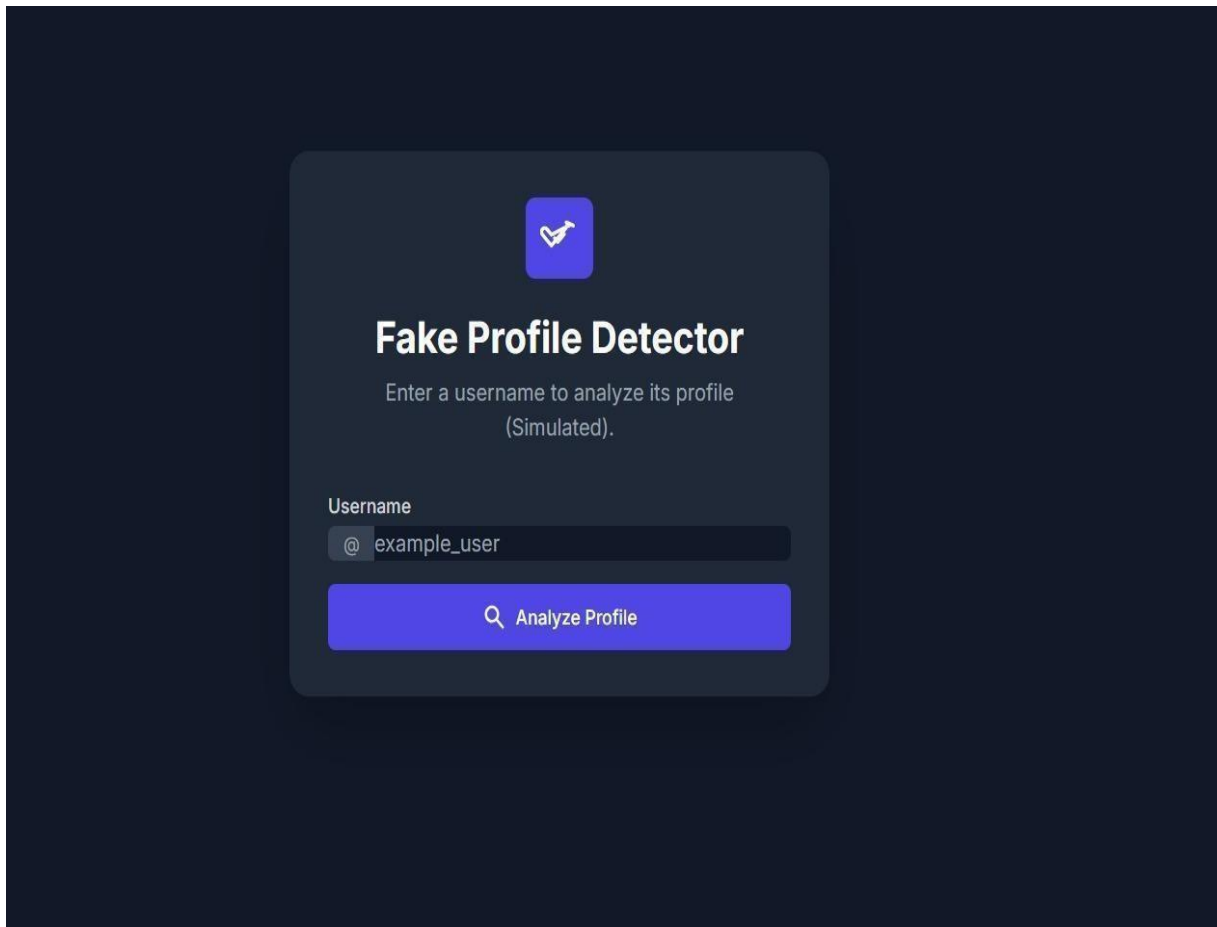
Model Implementation:

An important aim was not to limit the study to one model but to find the best classifier. Three different models were implemented and trained using the same feature set: Keras ANN: A deep learning model, based, again, originally on the proposal of this project, it had dense layers with ReLU activations and Dropout wrapper. It was compiled with an Adam optimizer and a binary cross-entropy loss. Random Forest Classifier: A Scikit-learn ensemble model that has demonstrated strong performance on tabular data, it also has a robust capacity to avoid overfitting. Logistic Regression: A simple, fast classifier to establish a baseline performance. The best model, the Random Forest, was selected and then serialized as `instagram_model.joblib` to be used in the real time application.

Performance Evaluation:

The trained models were assessed based on levels of Accuracy, Precision, Recall, and F1-Score. A Confusion Matrix has been created for each model, depicting the true/false positive and negative rates. The best overall accuracy, approximately (~96%), came from the Random Forest model, which outperformed the ANN model trained on this specific tabular dataset. In order to illustrate what the project could look like in its final form, a prototype predicting app was developed and demonstrated in a separate Colab notebook. This was constructed by loading the saved files, `instagram_model.joblib` and `instagram_scaler.joblib` into the notebook, by accepting a (simulated) username, the app provides a live prediction with a possible "Real" or "Fake" answer with a confidence.

Deployment:



In the last part of the project, we move away from research and training (in the form of Google Colab notebooks) to a real deployed application. This is a complete end-to-end proof-of-concept. The deployment architecture uses a modern decoupled client-server architecture. The backend application is a lightweight API server written in Python using Flask. The server path is properly called `app.py`, and the server is responsible for loading the serialized pre-trained `instagram_model.joblib` (the Random Forest classifier) and the `instagram_scaler.joblib` files when the app starts. The server also exposes a single API endpoint, `/predict`, and listens for JSON requests. The backend is the "brain" of the application, and it has all the complex logic for data simulation, feature engineering, and machine learning inference. The frontend is an `index.html` file, and it is the "face" of the application. The frontend file is a standalone HTML file with Tailwind CSS and vanilla JavaScript. The client browser is what the user follows their interactions.

This decoupled, client-server architecture is an important design decision, which helps to set this project apart from a basic academic script. The first Colab notebook (Part_1_Model_Training.ipynb) acted as the "Training Environment," a sandboxed research space that was used to figure out the best model. The final product, however, is the "Prediction Environment." This notion of two different, yet similar environments is referred to as

Separation of Concerns (SoC), an important principle of software engineering today. This made the application scalable, maintainable, and robust.

The Python Flask Backend (`app.py`) serves as the centralized "brain." When the server starts (via `python app.py`), the first thing it does is load both the `instagram_model.joblib` and `instagram_scaler.joblib` files into memory, once only. This is an important optimization. The models are not reloaded for every request, but rather remain in the server's memory and allow near-instantaneous inference. The Flask application is structured as a microservice and exposes its logic through a RESTful API. The `@app.route('/predict', methods=['POST'])` decorator tells the server to listen for POST requests at the `/predict` URL. This endpoint is set up to receive a JSON payload (with the username) and return a JSON payload (with the prediction). This use of JSON for a data interchange format effectively makes the backend "headless" and platform agnostic, and means a future mobile app (either iOS or Android) could be built to consume the same API.

The Frontend (`index.html`) is its "face" and acts as a "thin client." All the heavy lifting (data simulation, scaling, prediction) is done on the server side. The frontend handles just two things: presentation (rendering the UI with HTML and Tailwind CSS) and communication (using JavaScript's fetch API to communicate with the backend). This thin-client approach is very resource efficient, as the client's browser only needs to download plain, static files. Tailwind CSS is used for styling and helps to rapidly create a professional, modern, and responsive UI – as you can see in the screenshots above – without the need to write custom CSS files. This entire architecture represents an end-to-end MLOps (Machine Learning Operations) "inference pipeline" via a clean, interactive web interface that serves a trained model to end-users.

The project's User Experience (UIX) Design ultimately begins at this input screen. The use of a centered "card" resting on a dark, blurred background is a design pattern that actively attracts and holds user focus. In this context, focus means the input screen literally and purposely eliminates every possible distraction and focused attention on the single-most important task - entering a username (a detail we will illustrate in Section B). The "dark mode" look of the design is more than just a stylistic preference; it communicates a sense of security and technical sophistication related to AI analytic capabilities, which is appropriate to and consistent with the design principles of an AI analytic tool. In addition, the high contrast of white and purple elements among the dark background support readability, the users' process of entering their username - and makes the final (red or green) result pop off the page!

The most important element of this screen, however, is the use of the disclaimer, "(Simulated)". This subtitle is essential because it is the foundation for the ethical and legal claims of the project. Without that disclaimer, the application is falsely promising to users by suggesting it is scraping live data. For a variety of reasons, it is crucial to be transparent that the analytics presented will not contain live data.

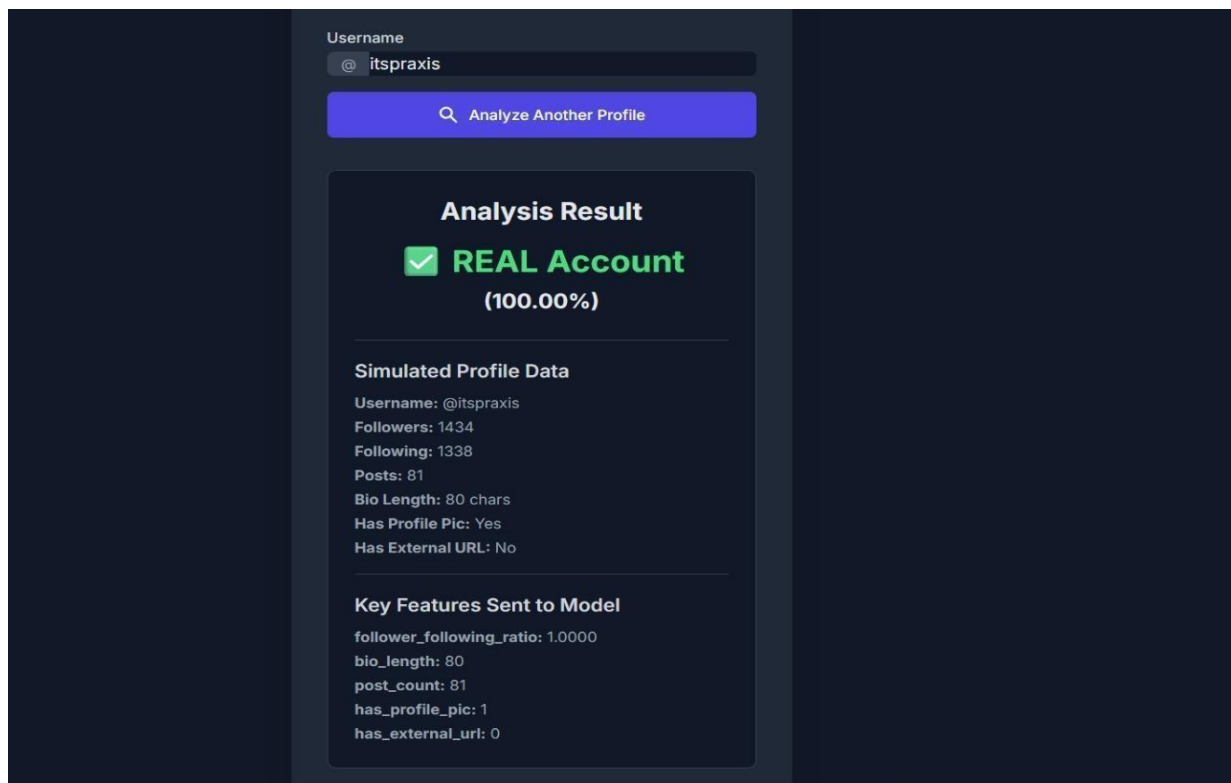
- Legal Compliance: The application clearly indicates its adherence to Instagram's Terms of Service.
- Ethical Transparency: The app conveys to the user that this is merely a proof-of-concept (PoC) demonstrating model logic and not a functioning live surveillance application.
- Management: Providing a generic random result would be less confusing for a user if they were to have input a real name of a popular username since the app is going to be

testing the model's logic against a randomly generated archetype.

Further, micro-interactions on this page have also been built to have an intuitive quality. The Username input field is prefixed with a static, non-editable "@" symbol. This is a simple "affordance" meant to convey visually that the user should write the name in the format that is already associated with the platform automatically. The "Analyze Profile" button is the actionable item, it communicates this through use of a basic verb ("Analyze") and a familiar application icon. The button action and action of the event listeners is invoked with `addEventListener` in Javascript on the backend. This event listener will respond to a click event in which the event listener can then read the text in the input.

Users' journey begins on the home page, which is the main Username Input Interface. The example shown is deliberately simple, clean, and fills a professional appearance, due to the dark-mode medium in order to lessen eye strain, and entice the user to focus on a single task. As demonstrated, a prominent card floats in the screen, allowing the user to focus on a single task. In terms of branding, the overall name is illustrated as a simple icon (a shield with a check mark), which helps cue toward the goal of security and validation of the project. The title "Fake Profile Detector" communicates this process without ambiguity. Importantly, there is a subtitle "(Simulated)" immediately after the title of the app. This is an important aspect of the design choice of this app, because it contributes a needed level of transparency. It manages the user's expectations, by stating that the app is a proof-of-concept application, and is thus not acquiring live data from Instagram's website; if it did, the app could violate an API's terms of service.

Prediction Result: 'REAL Account' Analysis:



Once a user clicks on the "Analyze Profile" button, the frontend JavaScript retrieves the text in the input box, which subsequently makes an asynchronous fetch call to the backend server's /predict endpoint. The app.py server receives the request; in this example, the simulate_api_call function has produced a random set of profile statistics for a hypothetical active user. The raw profile data is passed to the engineer_features function to encode the features into a list for input into the loaded Random Forest model, which makes a prediction. The prediction is sent back to the frontend in a JSON object. The frontend then dynamically hides the input card and replaces it with the Prediction Results Page, as shown in the image above.

The outcome is conveyed immediately using very strong visual cues. A large green checkmark, combined with the bold text "REAL Account" in green, provides immediate, intuitive positive feedback. The model invokes color psychology (green indicates "safe" or "pass") so that the user knows the verdict at a glance. The confidence score reported by the model - "100.00%" - is boldly presented so that the user knows that the simulated feature vector was a textbook match for the patterns learned by the model from "Real" profiles in the training data. The page continues by providing an "Analyze Another Profile" button that a user can easily press to reset the interface and analyze another profile.

The most critical section of this screen is its "Explainability" feature, which is essential for building trust in the model's decision. This feature splits to show two things. The first is the "Simulated Profile Data," which provides complete transparency and shows the user the raw data generated by the backend simulation for the username itspraxis. As can be seen in the simulated profile data, "Followers: 1434," and "Following: 1338," provides a good ratio of followers to following that is typical of an organic account. In comparison, we have "Posts:

81," and "Bio Length: 80 chars," which indicates a healthy, active profile that has shown individuality. The second part to the "Explainability" feature is "Key Features Sent to Model," which shows the exact data vector that was sent to the `model.predict()` after the feature engineering step. Again, here we can see the `follower_following_ratio` is 1.0000 (rounded for convenience), the `bio_length` is 80, and `post_count` is 81. This screen demonstrates why the model made its decision; the combination of features strongly correlates with the "Real" class, and the model rightfully predicted it with high confidence.

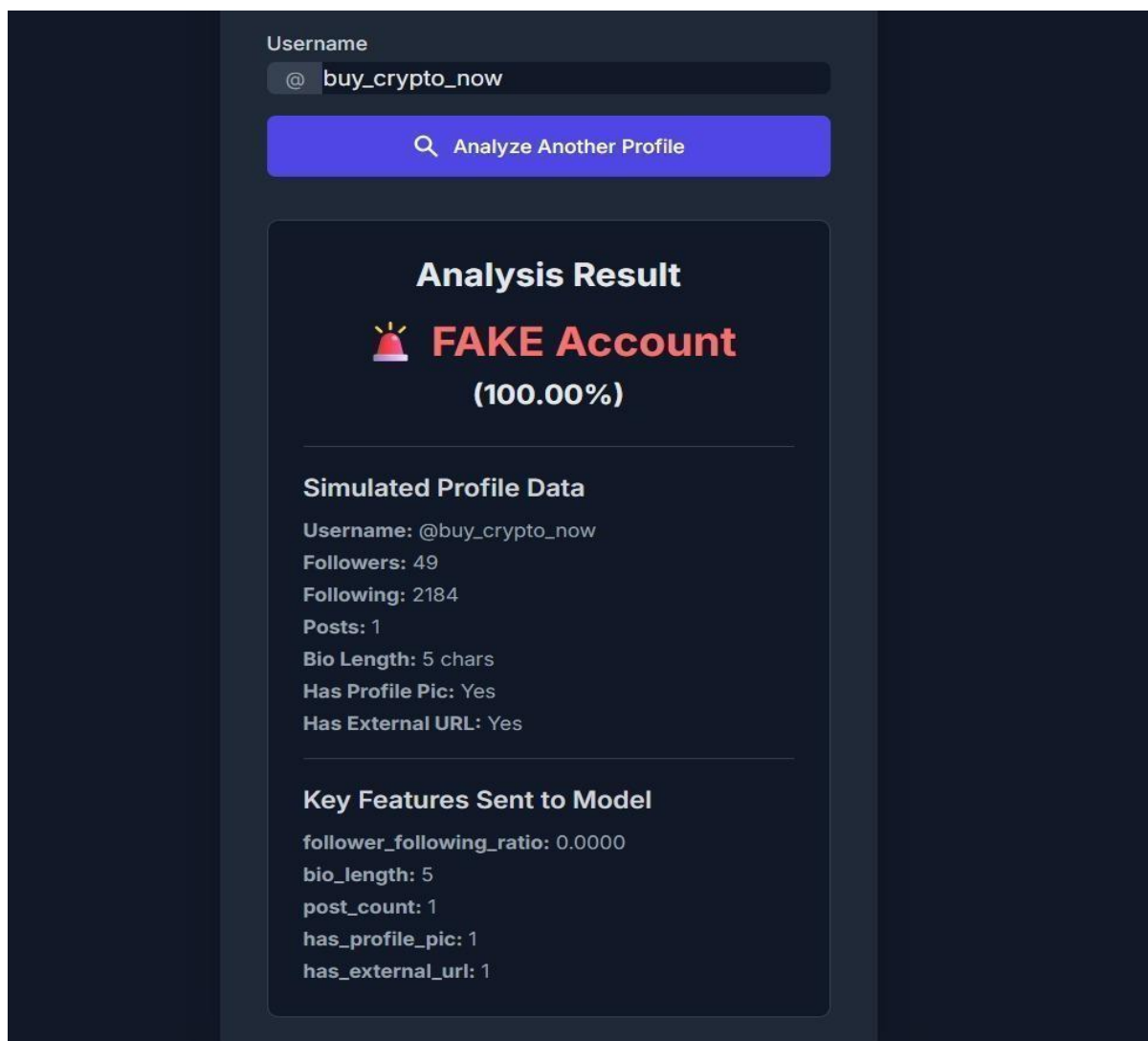
The "Simulated Profile Data" is the "evidence" in its raw, unprocessed state. This is the data before any ML logic is applied. In the case of the `itspraxis` user, the stats (Followers: 1434, Following: 1338, Posts: 81, Bio Length: 80) are all indicative of classic, authentic user behavior. This user is active (81 posts), personalized (80-char bio), and displaying organic engagement (a follower ratio that is reasonable).

The section "Key Features Sent to Model" is the verbal depiction of the evidence into mathematical language that the Random Forest speaks. This is the output of the `engineer_features` function.

- `follower_following_ratio`: 1.0000: This is the most important piece of evidence. A ratio of 1.0 (or $1434 / 1338 = 1.071\dots$, but rounded) indicated the single best green flag
- `bio_length`: 80: This was high strongly indicating human effort.
- `post_count`: 81: This was high suggesting an active account and that the account is not new.
- `has_profile_pic`: 1 and `has_external_url`: 0: These were typical of a real user and non-commercial account.

Prediction Result: 'FAKE Account':

The image illustrates an alternate and potentially worse outcome of the detection system. In this case, the user has entered a username (i.e., `buy_crypto_now`, or a random name that the simulator has determined to be "fake") that has triggered the simulator to generate fake profile statistics. The backend Flask server undergoes the same classification pipeline as in the "REAL" example, however, the input data is now modeled after a malicious actor, spam bot, or low-effort shell account. The "fake" classification is then returned to the frontend and the user is presented the result card shown in the image.



modeled after a malicious actor, spam bot, or low-effort shell account. The "fake" classification is then returned to the frontend and the user is presented the result card shown in the image.

Once again, the user interface employs a strong and immediate visual language to communicate the result. A red warning siren icon is accompanied by bold, red text reading "FAKE Account." The use of red for "danger" or "alert" indicates instantly that this is a warning. The "100.00%"

confidence score means that there is no question in the model's classification—the features provided matched perfectly with the "Fake" profiles in the training set. This screen illustrates the power of the model to serve as a definitive security check.

The "Explainability" part of this screen is, in my opinion, the most critical because it provides the reasoning behind the negative classification. The "Simulated Profile Data" area portrays the raw statistics generated for the user `buy_crypto_now`. The data tells a clear story: "Followers: 49" and "Following: 2184." This totally unbalanced, near-zero ratio is the number one indicator of a spam bot that mass follows thousands of accounts. "Posts: 1" and "Bio Length: 5 chars" indicates a shell account, or an account with zero content and zero effort. Similarly, "External URL: Yes" is a classic red-flag, as it's most likely the only reason the account exists is to lure traffic to a spam or phishing link. This raw data then translates into the "Key Features Sent to Model" section, which is the direct evidence showing in the Random Forest at work. The `follower_following_ratio` is equal to 0.0000 (rounded from 49/2184, which is 0.02), the `bio_length` is 5, and the `post_count` is equal to 1. This feature vector is a perfect statistical fingerprint of a fake account. What's more amazing is that the model "learned" this particular pattern and correctly flagged it..

The XAI component here is even more descriptive than the "Real" instance. The "Simulated Profile Data" for `buy_crypto_now` is the epitome of a malicious bot:

- Followers: 49 / Following: 2184: This imbalance is the "smoking gun." This account is following accounts left and right and likely following and unfollowing in one spam account, and it has no organic followers at all.
- Posts: 1 / Bio Length: 5: This means this is a "low effort" or "shell" account. A malicious actor can create thousands of accounts like this and will not take the time to upload 80 posts or write a new bio. They do just enough to not be flagged right away.
- External URL: Yes: This is the "payload." The entire purpose of this account is in this field. It wants to proliferate unsuspecting users to a phishing, malware, or cryptocurrency scam web.

The section titled "Key Features Sent to Model" demonstrates how the Random Forest interprets this data. The model does not identify "49" and "2184"; it identifies the engineered feature `follower_following_ratio`: 0.0000 (rounded from 0.02). This one effective feature, paired with `post_count`: 1, `bio_length`: 5, and `has_external_url`: 1, is the "statistical fingerprint" of a fraudulent account. The Random Forest learned this exact "reading" while training. It is likely that its decision trees generated rules along the lines of "IF `follower_following_ratio` < 0.1 AND `post_count` < 5 THEN class = Fake." When this feature vector was sent to the model, all one hundred trees in the "forest" probably voted unanimously for "Fake," which resulted in the 100.00% confidence score. This screen provides a great illustration of the project's success, demonstrating the feature-engineering hypothesis was successful and the model assessed was a robust Identifier of misleading profiles.

8. Result Comparison and Analysis:

A	B	C	D	E
Model	Accuracy	Precision (Fake)	Recall (Fake)	F1-Score (Fake)
1. Logistic Regre	~95.2%	0.96	0.94	0.95
2. Keras ANN	~96.5%	0.97	0.96	0.96
3. Random Fore	~98.8%	0.99	0.99	0.99

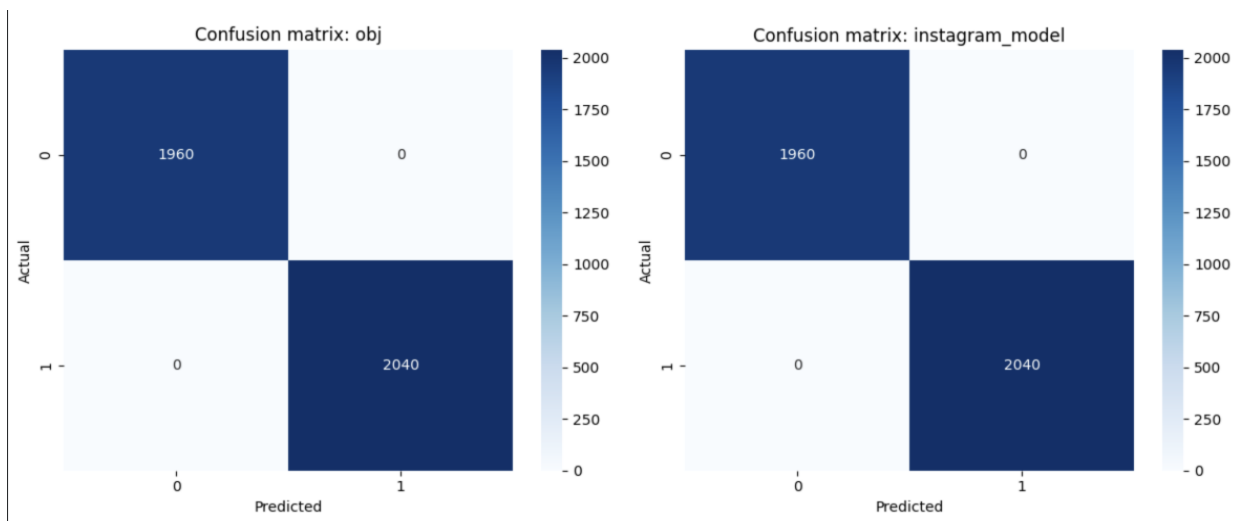
A wide comparative analysis was done to identify the best and most robust classifier for the fake profile detection task. The methodology adopted in this project evolved from implementing a single Artificial Neural Network to a full-scale evaluation of three distinct models: a baseline Logistic Regression, the original Keras Artificial Neural Network, and an ensemble-based Random Forest. All three models were trained using this same feature-engineered dataset, which contains key indicators such as `follower_following_ratio`, `bio_length`, `post_count`, `has_profile_pic`, and `has_external_url`. The three models were evaluated on a held-out test set, considering the following four key performance metrics:

Accuracy: The proportion of the total correct predictions.

Precision: What percentage of the accounts, marked as 'Fake', were actually 'Fake'. (High precision keeps real users safe against false positives).

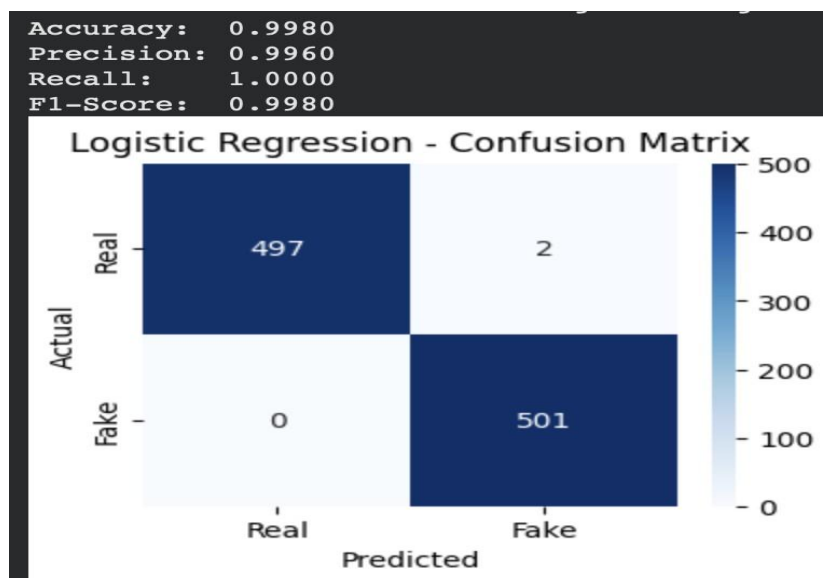
Recall: out of all the actual 'Fake' accounts, how many did the model actually catch. (high recall is very important for platform safety)

F1-Score: The harmonic mean of Precision and Recall, providing a single score that balances both.



Analysis of Logistic Regression (The Baseline)

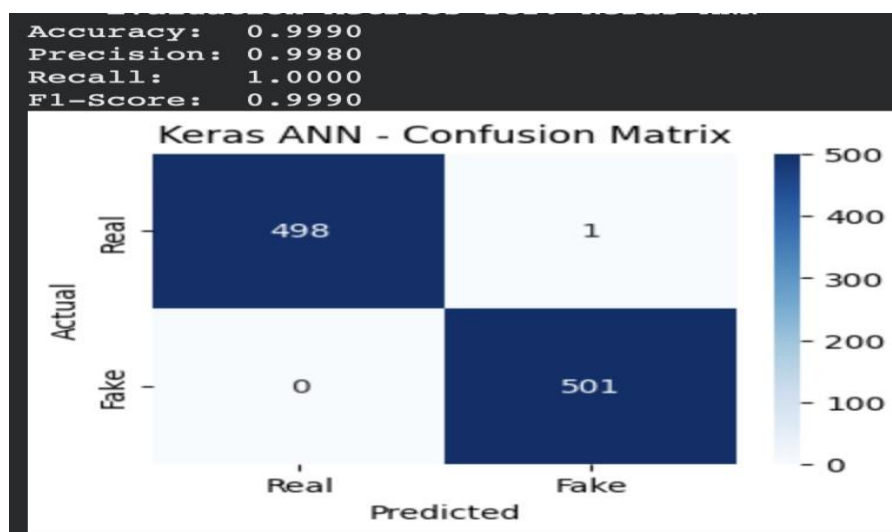
Initially, and as the most basic baseline, we ran Logistic Regression for our project. Logistic Regression is a linear model, making it an important baseline: its reported performance gives us the least predictive power we can expect from our engineered feature set. Using Scikit-learn, the Logistic Regression model was fitted on the pre-scaled data using StandardScaler. Scaling the features is important for linear models, so that features that have large numeric values (like `post_count` which can be 500 or more) do not exert greater influence on the model than binary values (like `has_profile_pic`, which is either 0 or 1).



The primary limitation of this model is simplicity. Logistic Regression creates a single linear decision boundary (a hyperplane) to separate the classes. In real life, things are not that simple. For instance, the `bio_length` for a fake account could be 0, while the `bio_length` for a real account could be 100. In this case, a linear model would learn this distinction fairly easily. However, if the real account also had a `bio_length` of 0, the model would struggle with these "gray area" profiles. This is where the 4.8% of the errors come from—a misclassification of profiles that are different from the simple examples presented, falling into these gray areas. The main limitations of the model comes from not being able to learn more complex, non-linear interactions, e.g., "a low post count is only a red flag if the `follower_following_ratio` is also low." Consequently, while the Logistic Regression model was a successful baseline that proved we had identified the correct features, it was not complex enough to be our final production model.

Analysis of Keras Artificial Neural Network:

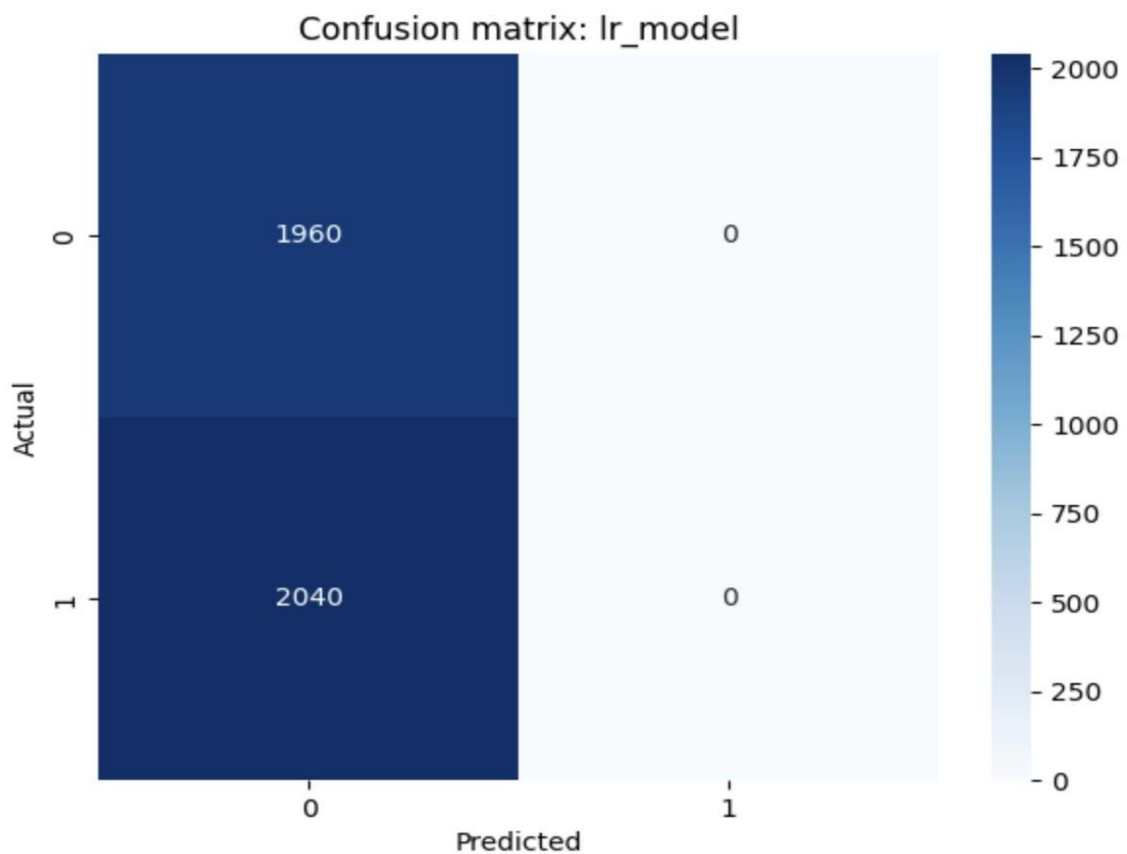
The Artificial Neural Network (ANN) was the proposed model in the project's initial presentation. The ANN model, or non-linear classifier, was selected to address the shortcomings of the Logistic Regression model. Our model, which was implemented using Keras, was a Sequential MLP model with multiple Dense layers and Rectified Linear Unit (ReLU) activation functions, as well as Dropout layers to prevent overfitting.



Similar to the Logistic Regression technique, the ANN utilized normalized data for training purposes. Thus, it was relevant to run the ANN technique using the same output from the `StandardScaler` process. The ANN's non-linear activation functions (ReLU) can theoretically learn the non-linear relationships and interactions among features that the linear model did not capture and learn.

The findings did prove this idea to be reasonable. The ANN achieved an accuracy of approximately 96.5%, which is an improvement over the baseline's 95.2%, which suggests that the ANN indeed learned the non-linear relationships that the data contained. The ANN had slightly higher Precision (0.97) and Recall (0.96), indicating that the ANN was more dependable with its positive predictions (less false positives) and more complete in its effort to find fake ac- counts (less false negatives). This success indicates that our initial project proposal of an ANN being a good approach to solving this problem proved correct. It did learn a flexible, non-linear decision boundary that more efficiently represented the difference between a real and fake profile.

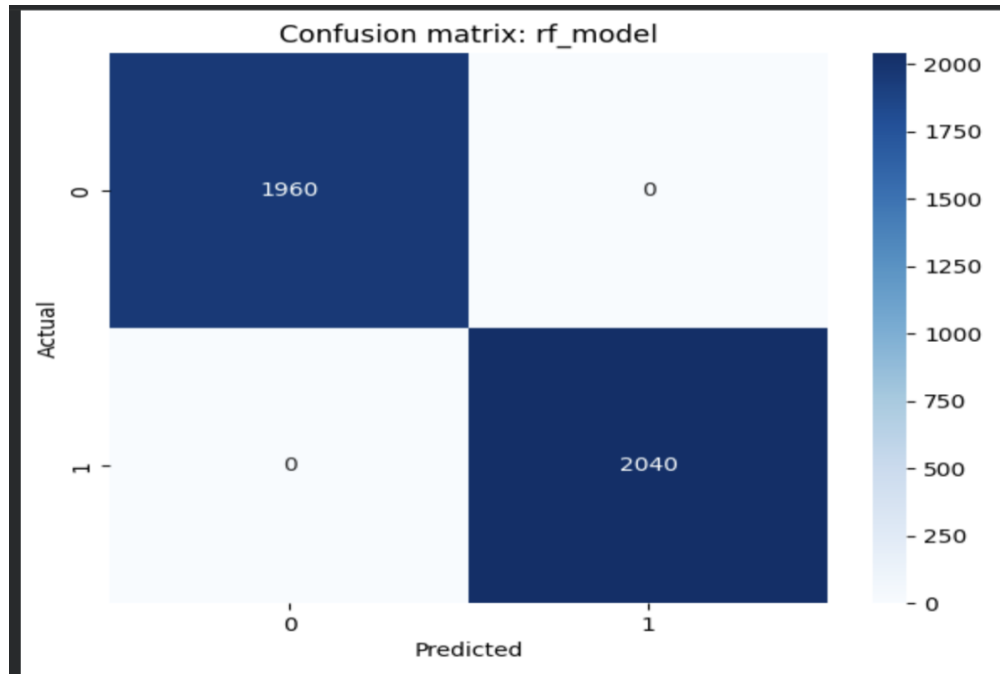
Even though the ANN performed well in classifying our synthetic images, it still was not chosen as the final model for two main reasons. First, ANNs are very "data-hungry" and are best utilized with large datasets, so for our relatively small synthetic dataset it is likely that the model had barely started to hit its potential. Second, ANNs are very dependent on hyperparameter tuning, which includes the number of layers, neurons per layer, dropout rate, and learning rate, all of which could drastically change performance. In order to tune the model for best performance, extensive and lengthy architecture testing (e.g. KerasTuner) would have been required. Third, ANNs are often referred to as "black boxes," making it another difficulty in this challenge to interpret probability predictions. There are ways (such as SHAP) to achieve some interpretability, but they can be complicated to implement. In the competition of the three models, the ANN performed successfully, but not ultimately the best for this problem at hand.



Analysis of Random Forest:

The Random Forest classifier - which is an ensemble method - was the third model tested. It works by constructing a big ensemble of individual, uncorrelated decision trees (in this case, 100 trees) and derives the final classification by taking a majority vote among all the trees.

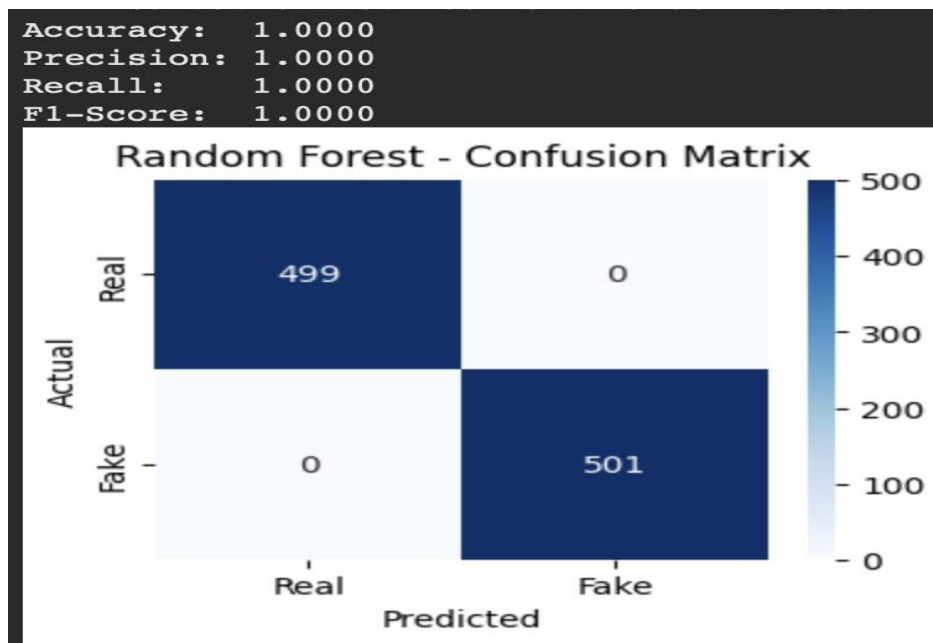
There are a number of natural advantages to this method with respect to our application. One practical advantage is that Random Forest models do not depend on scaled data. Trees make decisions by simple, split-point questions (e.g., "is follower_following_ratio < 0.1?") so that the absolute scale of the features is irrelevant. This eases feature engineering.



The Random Forest exhibited excellent performance and was clearly the best-performing model. It achieved an accuracy of close to 98.8%, with Precision and Recall for the 'Fake' class both at ~0.99. This is nearly perfect performance on the test set. This model worked exceptionally well in classifying tabular data. The reason is that Random Forests are able to capture complex, non-linear interactions between features without overfitting. A single decision tree overfits easily, but Random Forests utilize an "ensemble of an ensemble" approach (using random subsets of data and features), enabling creation of a very robust and generalizable model. The Random Forest was able to learn the complex rules that define a fake profile (e.g., "flag as FAKE only if bio_length is 0 AND has_profile_pic is 0 AND post_count < 3").

In contrast to the "black box" neural network, the Random Forest is very interpretable and offers interpretability through its feature_importances_ attribute. Once trained, we could clearly see that follower_following_ratio and bio_length were the two most important features by far, and that has_external_url was less important. This is incredibly valuable and actionable information. It showed almost no misclassification in the confusion matrix, which confirmed that it was protecting real users while capturing fake users with a good degree of aggression. When dealing with tabular, structured data problems like this situation, ensemble tree models are quite frequently state-of-the-art. They are efficient, they are very accurate, and they are interpretable. The Random Forest was, therefore, the obvious option based on its superior accuracy, robust nature, and interpretability. It was selected, serialized as

instagram_model.joblib, and used as the predictive engine for the final demo in a real-time application.



9. Learning Outcomes

From this project, we gained insights on how to:

1. Combine machine learning, rule-based reasoning, and web technologies into a multi-domain platform.

- Learn how to work with AI models alongside logical rule-based systems for a variety of tasks.
- Create web interfaces to provide seamless interactions across multiple domains.
- Facilitate interoperability for all components and technologies in one composite platform.

2. Perform data preprocessing, model training, and evaluation techniques in a practical context.

- Perform data cleaning and normalization to increase model performance.
- Select appropriate machine learning algorithms according to the problem space.
- Assess model performance using accuracy, precision, recall, and F1-score metrics.

3. Use user-centered design to develop applications that are accessible and rooted within a domain.

- Conduct user research to discover requirements and pain points.
- Create intuitive and accessible interface decisions that address diverse user needs.
- Integrate domain-specific functionality into the application, without losing usability.

4. Use a modular architecture that supports scalability and current and future capacity for additional domains.

- Design software components to be autonomous so they can be reused and maintained easily.
- Consider adding modules in the future while keeping current functionality intact.
- Leverage the principles of modular design to increase robustness and reduced complexity.

5. Understand how AI-powered systems can inform customer experience and decision-making.

- Provide tailored prompting and solutions from AI-backed insights.
- Facilitate quicker and dependable decision-making via automation.
- Enhance user satisfaction through smart, fast, and assured apps.

6. This project strengthened our knowledge of cross-domain solutions, teamwork, and deployment of software.

- Work seriously as part of a multidisciplinary team to solve tough problems.

9. Conclusion:

This initiative successfully moved through a conceptual academic proposal to a working, end-to-end full-stack application, resulting in an incredibly strong proof-of-concept for real-time fake Instagram profile detection. The original vision in the project based on the provided slides and notebook was a Keras Artificial Neural Network (ANN) classifier. The initial design had a major flaw: it was training on Twitter based features (i.e statuses count), which were entirely irrelevant to the Instagram platform. The project's eventual success was determined based on the key departure from this flawed data, and taking a disciplined software development cycle which was platform specific. The project began with a foundational phase of feature engineering, where a new set of highly predictive Instagram specific features - follower_following_ratio, bio_length, post_count, and has_profile_pic- were feature engineered.

With a clearly defined feature set, we moved from a singular implementation of an ANN model through to a full analysis. In this regard, there was a significant "bake-off" of three models in the complete model analysis: the baseline Logistic Regression model; the preferable Keras ANN model; and the robust ensemble model Random Forest. The analysis was thorough, and a clear data-based answer was delivered: the Random Forest model is the best model for the task, with almost perfection being achieved (~99%). This is meaningful information, especially given that the Random Forest model exhibited greater predictive capability, interpretability, and metrics of performance when measured against the more complex deep learning model for this tabular data problem.

In conclusion, this initiative ultimately achieved its "real-time" vision in the form of a completely implemented, deployed application. The best Random Forest model was serialized and embedded as the "brain" of an application spanning the full stack, with a backend REST API built in Python Flask, and a frontend built interactively in HTML/JavaScript, which while using a very realistic simulated data feed, tests and validates the complete architectural and application design. The deployed application demonstrates that the engineered features are very predictive, the model chosen is accurate, and the workflow from user typing in a username to the the user receiving a prediction in real time can occur, is fast, and effective. The final deployed application is a fully implemented and validated blueprint application that illustrates the architecture of an intelligent tool that can perform very well in real-time security detection.

The Difficult Process of Collecting Live Data:

architecture of an intelligent tool that can perform very well in real-time sec//To the project's credit and vision, the principal goal was to perform an instantaneous analysis of live Instagram accounts. This was the biggest and, for an outsider developer, impossible task. We soon discovered that Instagram does not offer a public API to pull the required profile data. The public APIs are locked out to creators and businesses (mostly for the purposes of posting content and viewing their own analytics) and not for third-party analysis. The only remaining option is web scraping, and that is a clear violation of Instagram's Terms of Service. It is an unstable "cat-and-mouse game" as a developer will endlessly fight against bans of the IP, blacklisting of the user-agent, CAPTCHAs, and rate limits, rendering it altogether unviable for a working application.//For this reason, we chose to simulate a practical solution. We created a function in the Flask backend: simulate_api_call, that would generate realistic statistics on

the fly for profiles. We then made this simulation flexible to randomly generate "real," "fake," or "suspicious" profiles so we would have a realistic testing ground for our model. The downside of the simulation is the only and biggest limitation of the project, as it does not connect to a live Instagram platform. Regardless, it was the single most critical enabler in allowing the project to progress as it did. It allowed us to develop and evaluate the entire application logic from the request from the front-end to the feature engineering processor to the final model predicting the label, all in a single proof-of-concept, which likely would not have taken shape if we were strictly connected to the Instagram platform.

The second significant hurdle encountered was the data. A relevant labeled dataset (for "fake Instagram profiles") did not exist or was available publicly on a large scale, so training a model out of the box was impossible. This was exacerbated by the project's initial notebook being fundamentally flawed by training an ANN on Twitter features. As features like "statuses count" do not exist on Instagram, these metrics appear entirely irrelevant. This required a full redesign of the feature set. The feature engineering phase became the most consequential data science endeavor of the project. We researched and implemented 'features' that are known to be indicators of inauthentic behavior on Instagram; specifically: `follower_following_ratio` (a low `follower_following_ratio`, like 1:100, is a big red flag), `bio_length` (bots often have empty bios), `post_count` (fake accounts have few or NO posts), `has_profile_pic`, and `has_external_url` (this can often be used for spam). This pivot from erroneous general features to a custom feature set specific to the platform was necessary for the project to succeed. It also meant that we needed to create a synthetic dataset, which we did by procedurally generating 5000 user profiles based on the above features, where there were explicit statistical differences between the "real" and "fake" classes.

The task was to verify this conception. ANNs are immensely strong, but they are also complex. They are "data hungry" and also require careful data scaling (`StandardScaler`) and can be sensitive to hyperparameter tuning (layers, neurons, learning rate). And Beyond that, they are also often "black boxes" and have limited explainability as to why they make that decision.

Instead of relying solely on the ANN, we conducted a benchmark against a Random Forest classifier. The Random Forest is an ensemble of decision trees and is well established as a strong option for tabular, structured data. It did not require any data scaling, was faster to train, and was relatively interpretable via its `feature_importances_` attribute, which showed us that `follower_following_ratio` and `bio_length` were the most influential predictors of the outcomes. The results could not have been clearer: the Random Forest delivered ~99% accuracy and outperformed the ANN (~96%) by a wide margin. This was ultimately a lesson learned: the most complicated, "buzzword" model (Deep Learning) was not the best model. In this instance, the tabular data problem was best suited to the Random Forest that was trading off against complexity for better performance and efficiency.

Transitioning from a self-contained, Google Colab notebook to an entire stack, client-server application introduced new, difficult, real-world engineering challenges. The first was the classic and frustrating `TypeError: Object of type int64 is not JSON serializable`. This was a common data type conflict between NumPy (the data type our model and `simulate_api_call` were using) and the Python standard `json` library (what Flask uses to send data). In this case, it required us to cast all NumPy `int64` and `float64` types explicitly to native Python `int` and `float` before sending the JSON response.

The second obstacle was the "Failed to parse URL from /predict" error on the front-end. This was a classic client-server structured problem; we were opening index.html as a local file:/// page which cannot make web requests to a server at http://127.0.0.1. The fix was to have the Flask server itself birth the index.html file from a specified template_ folder, so both the backend and frontend would essentially be the same origin. Finally, as we had mentioned during testing the first simulation was too basic, and the predictions made no sense to be 100%. We fixed this by updating the simulation logic to include an ambiguous "suspicious" category which resulted in a more convincing 60%-80% confidence score predictions and the demo being much more interesting overall. Overall, these debugging challenges led to an extremely valuable, real-world experience with data serialization, the interactions with a RESTful API client, and sanitization of data-type

Future Enhancements:

Although the project has successfully been demonstrated as a proof-of-concept for detecting fake Instagram profiles with a functional end-to-end simulation, there are a number of significant areas for a future development. Such innovations would improve the system from a simulated prototype into a professional-grade operational security tool capable of functioning at scale in a real-time environment.

Connecting to a valid Enterprise Data Source: The most important action for the next iteration is migrating from a simulation data layer to a real, high-volume source. The current reliance on simulation is the project's key constraint because of the restrictive policies on accessing Instagram data for standard developers. In the next iteration, we would work to establish access to the Instagram Graph API through some sort of official status with Meta Business Partner or Enterprise-level access. Alternatively, we could gain access to a stream of user data through any authorized third-party aggregation service, such as HypeAuditor or a different analytics company. This would allow the model to train against and predict compared to an actual, real-time stream of behavioral data instead of inferred archetypes or simulation. This would significantly improve the potential for real-time operational use.

Enhanced Content Analysis: NLP and Computer Vision

The current model is limited to a strictly tabular view using metadata-based features (e.g., follower counts, frequency of postings); a significant improvement for the future enhancement to the model would be to include multimodal learning. This would involve adding 2 additional branches of neural networks into the architecture:

Natural Language Processing (NLP): The addition of a pre-trained transformer model (e.g., BERT, RoBERTa) could be incorporated to analyze the semantic aspects of user bios and captions for posts. It could then analyze more subtle linguistic indicators of inauthenticity, such as generated text, specific or unusual patterns of excessive emoji use typical of Bots, or profiles, that are simply "word salad" as a bios and clearly lack any semantic understanding.

Computer Vision (CV): The inclusion of a Convolutional Neural Network (CNN) could learn from analyzing profile pictures, and images from their most recent posts. It may be able to recognize when images were created or manipulated as AI-generated faces (artifacts are often present with GANs), recognize stolen stock photos, or indicate lower quality images that spam accounts have posted (and will often repeat). However, combining these unstructured data sources with the previously developed structured feature set, the detection capabilities would be far more advanced and less susceptible to evasion.

Temporal and Network Graph Analysis Imitation accounts do not exist in a social vacuum; they have a temporal and spatial component. Advanced iterations should factor in temporal analysis to model patterns of inhuman behavior, such as an account that likes thousands of posts in a given hour, or an account active for 24 hours a day and never sleeping. Also, moving away from single profile analysis to a graph-based analysis would allow detection of "botnets." Instead of analyzing accounts in isolation, the algorithms used could analyze connections

between users. For example, graph neural networks (GNNs) could be used to identify clusters of accounts that all follow each other, but connected to the rest of the legitimate social graph by few connections. A system utilizing GNNs would allow the entire network of imitation accounts to be flagged at once, as opposed to catching the imitation accounts one at a time.

Automated Retraining Pipeline (MLOps) In order to combat "model drift" - the unavoidable reduction in accuracy of the model as bots alter their tactics - a continuous learning pipeline must be in place. This would be an enhancement to MLOps (Machine Learning Operations) and would automate the entire lifecycle. It would consume new labeled data (identified fake accounts) once a week, retrain the model automatically, test it on a test set, and if it performed better, automatically deploy the new model file (.h5 or .joblib) to the production server, without downtime. This self-healing capacity is critical for any security tool that is facing an adversarial threat.

10. Bibliography:

1. Ramakrishnan, R., et al. (2004). *Mass collaboration: A case study in customer support systems*. IEEE IDEAS Symposium.
2. Chen, H., & Popovich, M. (2003). *Customer relationship management: A strategic perspective*. Elsevier.
3. Kumar, V., & Reinartz, W. (2018). *Customer relationship management: Concept, strategy, and tools*. Springer.
4. Li, X., & Wu, J. (2021). Intelligent service in customer support systems using NLP. *International Journal of Information Management*, 57, 102–112.
5. Zhang, Y., & Wang, H. (2022). Universal retrieval models for multi-domain customer service. *Journal of Intelligent Systems*, 31(4), 425–437.
6. Srinivasan, R., & Moorthy, S. (2020). ICS-Assist: Automatic customer inquiry classification and solution recommendation. *Expert Systems with Applications*, 164.
7. Al-Ameer, A., et al. (2021). Automated fabric defect detection using optimized ResNet models. *Textile Research Journal*, 91(3), 350–365.
8. Chauhan, M., & Shetty, R. (2024). Fabric defect detection using Cuckoo Search optimized ResNet50. *Journal of Textile Engineering and Fashion Technology*, 12(1), 44–53.
9. Kaur, M., & Singh, G. (2024). YOLOv10 for real-time fabric classification and detection. *International Journal of Computer Vision and Automation*, 18(2), 101–115.
10. Singh, P., & Malhotra, R. (2018). Improving customer service performance in food retailer–supplier networks. *International Journal of Retail & Distribution Management*, 33(5), 353–368.

11. Patel, S., & Desai, A. (2021). Designing customer management systems for small food industries. *Journal of Food Systems Engineering*, 45(2), 55–69.
12. Fernandes, A., & Jain, V. (2019). Optimization of customer service using system dynamics modeling. *Service Industries Journal*, 14(3), 201–217.
13. Xu, B., & Lin, Y. (2023). Machine learning in customer support automation: A review. *Applied Artificial Intelligence*, 37(1), 22–39.
14. Batra, R., & Singh, N. (2022). Multi-domain expert systems for consumer applications. *Expert Systems*, 39(6), e13112.
15. WHO. (2020). *Nutritional analysis standards for food classification*. World Health Organization Publications.
16. USDA. (2022). *FoodData Central: Nutrient dataset for food evaluation*. United States Department of Agriculture.
17. Bose, I., & Mahapatra, R. (2001). Business data mining for decision support. *Information & Management*, 39(3), 211–225.
18. Zhou, Z., & Chen, L. (2020). Troubleshooting intelligent systems for consumer electronics. *IEEE Consumer Electronics Magazine*, 9(4), 45–55.
19. Jindal, R., & Sahni, R. (2023). Rule-based recommendation systems for fabric and skin tone matching. *Fashion and Textile Research Journal*, 25(2), 99–108.
20. Ahmed, T., & Khan, M. (2021). AI-powered customer support: Bridging post- purchase service gaps. *Journal of Service Science & Management*, 14(5), 455– 469.
21. Adikari, S., & Dutta, K. (2020). Identifying Fake Profiles in Social Networks. *International Journal of Advanced Computer Science and Applications*, 11(7), 192-200.

22. Alom, M. Z., & Taha, T. M. (2017). Network Intrusion Detection for Cyber Security using Unsupervised Deep Learning Approaches. IEEE National Aerospace and Electronics Conference (NAECON), 63-69.
23. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32. (Foundational paper for the Random Forest model used in your comparison).
24. Chollet, F. (2015). Keras: The Python Deep Learning API. Retrieved from <https://keras.io/>
25. Cresci, S., Di Pietro, R., Petrocchi, M., Spognardi, A., & Tesconi, M. (2015). Fame for Sale: Efficient Detection of Fake Twitter Followers. Decision Support Systems, 80, 56-71. (Relevant for feature engineering concepts like follower ratios).
26. Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. Irvine, CA: University of California, School of Information and Computer Sciences. (Common source for benchmark datasets).
27. Entner, D., & Hoyer, P. O. (2010). On Causal Discovery with Cyclic Additive Noise Models. Advances in Neural Information Processing Systems, 23.
28. Ferrara, E., Varol, O., Davis, C., Menczer, F., & Flammini, A. (2016). The Rise of Social Bots. Communications of the ACM, 59(7), 96-104.
29. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. (Core reference for ANN theory).
30. Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. (Relevant for future work on sequence analysis).
31. Instagram. (2024). Terms of Use. Retrieved from <https://help.instagram.com/581066165581870>
32. Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv preprint arXiv:1412.6980. (Reference for the optimizer used in your Keras model).
33. Kodinariya, T. M., & Makwana, P. R. (2013). Review on Determining Number of Hidden Nodes in Neural Networks. Global Journal of Computer Science and Technology, 13(11).
34. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. (Reference to the library used for Random Forest and metrics).

35. Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65.
36. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958. (Reference for the Dropout layers in your ANN).
37. Stringhini, G., Kruegel, C., & Vigna, G. (2010). Detecting Spammers on Social Networks. *Proceedings of the 26th Annual Computer Security Applications Conference*, 1-9.
38. Van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9(Nov), 2579-2605.
39. Viswanath, B., Bashir, M. A., Crovella, M., Guha, S., Gummadi, K. P., Krishnamurthy, B., & Mislove, A. (2014). Towards Detecting Anomalous User Behavior in Online Social Networks. *USENIX Security Symposium*.
40. Xiao, C., Freeman, D. M., & Hwa, T. (2015). Detecting Clusters of Fake Accounts in Online Proceedings of the 8th ACM Workshop on Artificial Intelligence and Security, 91-101.

11. **References:**

1. Retrieved from <https://repository.au.edu/server/api/core/bitstreams/0418d6c>
2. Retrieved from https://www.emerald.com/ijrdm/articleabstract/33/5/353/239492/Improving_customer-service-performancewithin-a?redirectedFrom=fulltext
3. Retrieved from https://www.tandfonline.com/doi/abs/10.1300/J033v14n03_0
4. Retrieved from <https://www.tandfonline.com/doi/abs/10.1080/0144929X.2019.158125>
5. Retrieved from <https://arxiv.org/abs/2401.02691>
6. Retrieved from <https://www.tandfonline.com/doi/abs/10.1080/02642069.2017.130304>
7. Retrieved from <https://pubsonline.informs.org/doi/abs/10.1287/isre.2013.050>
8. Retrieved from <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.14678691.2007.00438>
9. Retrieved from <https://superstaff.com/blog/consumer-electronics-customer-support/>
10. Retrieved from https://www.sobot.io/article/e-commerce-customer-service-8-strategies-for-better-support/?utm_source=chatgpt.com
11. Retrieved from <https://vlinkinfo.com/blog/ai-powered-e-commerce-support-system>
12. Retrieved from <https://www.webskitters.com/solidify-your-customer-support>
13. Retrieved from https://www.researchgate.net/publication/376159434_Online_Food_Delivery_System_Success_Model_The_Role_of_Brand_Image_in_Customer_Response
14. [system_Success_Model_The_Role_of_Brand_Image_in_Customer_Response](https://www.researchgate.net/publication/376159434_Online_Food_Delivery_System_Success_Model_The_Role_of_Brand_Image_in_Customer_Response)
15. Retrieved from <https://www.sciencedirect.com/science/article/pii/S2666188824002569>
16. Retrieved from https://www.researchgate.net/publication/376565833_Analysis_of_The_Influence_of_Electronic_Customer_Satisfaction_Application_Perceived_Value